

# Adaptively Secure Threshold ElGamal Decryption from DDH

Sourav Das

Ling Ren

Ziling Yang

University of Illinois at Urbana-Champaign  
{souravd2, renling, zilingy2}@illinois.edu

**Abstract.** Threshold decryption schemes allow a group of decryptors, each holding a private key share, to jointly decrypt ciphertexts. Over the years, numerous threshold decryption schemes have been proposed for applications such as secure data storage, internet auctions, and voting, and recently as a tool to protect against miner-extractable value attacks in blockchain. Despite the importance and popularity of threshold decryption, many natural and practical threshold decryption schemes have only been proven secure against static adversaries.

In this paper, we present two threshold decryption schemes that withstand malicious adaptive corruption. Our first scheme is based on the standard ElGamal encryption scheme and is secure against chosen plaintext attack (CPA). Our second scheme, based on the chosen ciphertext attack (CCA) secure Shoup-Gennaro encryption scheme, is also CCA secure. Both of our schemes have non-interactive decryption protocols and comparable efficiency to their static secure counterparts. Building on the technique introduced by Das and Ren (CRYPTO 2024), our threshold ElGamal decryption scheme relies on the hardness of Decisional Diffie-Hellman and the random oracle model.

**Keywords:** Threshold cryptography · adaptive security · chosen plaintext security · chosen ciphertext security · public key encryption.

## 1 Introduction

Threshold public-key decryption schemes [Des88, Des93] distribute the private decryption key among a group of decryptors. To decrypt a ciphertext, each decryptor publishes a partial decryption share computed from their private key. Any threshold number of correct partial decryption shares can recover the plaintext, while fewer shares reveal no information about the plaintext.

One simple and popular threshold decryption scheme is the threshold ElGamal decryption scheme, first introduced by Desmedt [Des93] and later refined by Cramer, Gennaro, and Schoenmakers [CGS97], which adapts the classic (non-threshold) ElGamal encryption scheme [ElG85]. The ElGamal threshold decryption scheme preserves the advantages of ElGamal encryption, such as being simple and relying only on the Decisional Diffie-Hellman (DDH) assumption. Also inherited from its non-threshold counterpart, threshold ElGamal encryption is only secure against chosen plaintext attacks (CPA), where the attacker can only obtain encryptions of plaintexts of its choosing.

A stronger security notion is chosen ciphertext security (CCA), where the attacker can additionally obtain decryptions of ciphertexts of its choosing. One of the first threshold CCA secure decryption schemes is Canetti and Goldwasser [CG99], which modified the Cramer-Shoup CCA (non-threshold) encryption. Their scheme requires decryptors to hold a large amount of shared randomness and to interact during decryption for robustness. Shoup and Gennaro [SG02] point out that CCA-secure threshold decryption schemes require partial decryptions to be publicly verifiable and constructed the first non-interactive CCA-secure threshold decryption scheme. Their scheme is based on ElGamal and DDH in the random oracle model (ROM), and uses a non-interactive zero-knowledge proof to achieve public verifiability of partial decryptions. Boneh et al. [BBH06] proposed the first non-interactive CCA-secure threshold decryption scheme that does not rely on the ROM; it uses bilinear pairing for public verifiability. Other CCA-secure threshold decryption schemes include [Wee11, DK05, MK02].

All of the above schemes assume a static adversary. A static adversary is one that decides the set of decryptors to corrupt before the execution of the protocol. A more powerful and more realistic adversary is

an adaptive one that can adapt its corruption strategy according to its view during the protocol execution and can corrupt any party at any point of the execution. Many existing schemes that achieve adaptive security and CCA security require interaction during the decryption process or/and an offline phase [CGJ<sup>+</sup>99, FMY99a, FMY99b, JL00]. Only a few threshold decryption schemes exist that are non-interactive and CCA-secure in the adaptive corruption setting. Libert and Yung [LY12, LY13] proposed such schemes using bilinear pairings, and Devevey et al. [DLN<sup>+</sup>21] introduced a lattice-based construction.

**Our Contribution.** In this paper, we propose two non-interactive threshold decryption schemes secure against adaptive corruptions. Both schemes rely on the DDH assumption in the Random Oracle Model.

1. A CPA-secure threshold decryption scheme compatible with the non-threshold ElGamal, i.e., the encryption algorithm is identical to classic ElGamal.
2. A CCA-secure threshold decryption scheme based on the Shoup-Gennaro, which is a natural CCA version of threshold ElGamal and also requires DDH and ROM.

These are the first non-interactive threshold decryption schemes secure against adaptive corruptions based on the standard DDH assumption in the ROM.

## 1.1 Technical Overview

The technical overview is divided into two parts. First we provide an overview of our CPA-secure scheme based on the standard ElGamal decryption scheme. Next, we provide an overview of our CCA-secure scheme.

**Threshold CPA secure decryption scheme.** We first describe the standard ElGamal encryption [ELG85]. The ElGamal encryption of a message  $m$  is  $\text{ctxt} = (c, u) = (m \cdot \text{pk}^r, g^r)$  where  $r$  is secret randomness chosen by the encryptor and  $\text{pk}$  is the public key in the form of  $\text{pk} = g^x$ . To recover the message, a decryptor with the decryption key  $x$ , computes the message as  $m = c/u^x$ .

In the static secure threshold ElGamal decryption scheme [Des93, CGS97], the  $i$ -th decryptor holds a Shamir secret share of the decryption key  $x$ . Let  $x_i$  be the secret share of decryptor  $i$ . To decrypt the ciphertext  $\text{ctxt}$ , each decryptor  $i$  publishes its decryption share  $u^{x_i}$ . A combiner recovers the plaintext from a threshold number of valid decryption shares by interpolating in the exponent. In the threshold ElGamal decryption scheme, we publish a per party public key  $\text{pk}_i = g^{x_i}$  to check the validity of partial decryptions.

The static security of threshold ElGamal decryption relies on a reduction of breaking the IND-TCPA security of the decryption scheme to breaking the DDH assumption. More concretely, when the reduction adversary against DDH receives a tuple of  $(g^\alpha, g^\beta, g^\zeta)$ , it embeds  $g^\alpha$  into the public key, i.e.,  $\text{pk} = g^\alpha$ . Then the reduction adversary must send the per party public key,  $\text{pk}_i$  of each party that interpolates to  $\text{pk}$ , along with the public key  $\text{pk}$ . Notice that each  $\text{pk}_i$  perfectly binds party  $i$ 's secret key (only one  $\text{sk}_i$  for a given  $\text{pk}_i$ ). Therefore, upon corruption of party  $i$ , the reduction adversary must solve for  $\text{sk}_i$  by finding the discrete log of  $\text{pk}_i$ . In the static corruption setting, the reduction adversary circumvents solving the discrete log problem. The reduction adversary, knowing the set of corrupt parties beforehand, first randomly samples the secret keys of the corrupt parties and computes the per party public keys of the corrupt parties from their secret keys. Lastly, it “interpolates in the exponent” to generate consistent per party public keys for honest parties.

But this is exactly the step that creates the difficulties for proving adaptive security. The reduction adversary does not know which parties will be corrupted at the time of computing the per-party public keys since an adaptive adversary can corrupt any party at any time during the execution of the protocol.

Upon adaptive corruption of party  $i$ , the reduction adversary must provide  $\text{sk}_i$  that is consistent with  $\text{pk}_i$  by solving the discrete logarithm of  $\text{pk}_i$ , which is known to be difficult. The standard threshold ElGamal scheme [Des93], along with most other statically secure threshold decryption schemes, does not allow the reduction adversary to consistently simulate the view of all parties.

The recent work of Das and Ren [DR24] introduced a proof technique for an adaptively secure threshold BLS signature scheme. In our paper, we utilize such a technique to construct a new threshold decryption scheme and prove it adaptively secure. Observe that the difficulty of proving adaptive security lies in providing correct Shamir secret shares of  $\alpha$  when only  $g^\alpha$  is known. This difficulty arises from embedding part of the DDH challenge,  $g^\alpha$  into the public key and therefore the aggregate secret key,  $\alpha$  is unknown. Further observe

that if the aggregate secret key,  $\alpha$  is known, then simulating all secret keys and per party public keys becomes trivial. In Das and Ren's proof technique, the reduction adversary embeds the challenge elsewhere and knows the aggregate secret key in the clear, thus avoiding the aforementioned difficulty. To embed  $g^\alpha$  into the public key and provide a consistent view, we follow Das and Ren's approach and modify the standard ElGamal public key to  $\text{pk} = g^x h^y$ . The additional group element  $h$  in our public key allows the reduction adversary to set  $h = g^\alpha$  and locally sample the decryption key  $(x, y)$ . In our scheme, we use  $y = 0$  so that the encryption protocol remains unchanged from the standard ElGamal scheme. However, in the security proof against adaptive corruptions, the reduction adversary, when interacting with the adaptive adversary, will run an instance of our protocol with a rigged public key with  $y \neq 0$ .

The final part of our protocol is how the reduction adversary simulates consistent decryption shares under the rigged public key. The original threshold ElGamal's partial decryption protocol will produce a ciphertext  $m' = m \cdot (g^{rx} h^{ry}) / g^{rx} = m \cdot h^{ry}$  where  $r$  is the randomness chosen in the encryption protocol. Notice that  $m'$  is not the underlying plaintext. Therefore, the reduction adversary cannot simulate a correct view with the original decryption protocol. Thus, we use a programmed random oracle to recover the plaintext. Our modified partial decryption protocol will output a partial decryption of  $\hat{m}_i = g^{x_i r} \cdot \text{H}(\text{ctxt})^{y_i}$  instead of the original  $g^{x_i r}$ . Then, by programming  $\text{H}(\text{ctxt}) = h^r$ , we successfully recover the correct plaintext. In our modified scheme, the combiner computes  $m \cdot \text{pk}^r / \hat{m} = m \cdot (g^{rx} h^{ry}) / g^{rx} h^{yr} = m$ , where  $\hat{m}$  is the aggregated partial decryption. Notice that in the real-world execution of our scheme, when  $y = 0$ , the partial decryptions also correctly recover the underlying plaintext  $m$ .

**Threshold CCA secure decryption scheme.** Our threshold CCA secure decryption scheme is built upon Shoup-Gennaro's TDH2 \* [SG02] scheme, which builds upon the basic threshold ElGamal scheme and achieves CCA security by including extra elements that allow the reduction simulator to simulate decryption queries. The classical ElGamal encryption is of the form  $(u, c) = (g^r, g^r) \cdot m$ . The extra elements take the form of  $\bar{u} = \bar{g}^r$  where  $\bar{g}$  is a generator and a NIZK proof of equality of discrete logarithm of  $u$  and  $\bar{u}$ . Shoup-Gennaro's scheme allows straight-line extraction of (with knowledge of a trapdoor) sufficient information for answering the adversary's decryption queries.

To encrypt a message  $m$ , the encryptor computes

$$u := g^r, c := m \cdot \text{pk}^r, w := g^s, \bar{u} := \bar{g}^r, \bar{w} := \bar{g}^s, e := \text{H}(c, u, w, \bar{u}, \bar{w}), f := s + re$$

The ciphertext is  $\text{ctxt} := (c, u, \bar{u}, e, f)$ . Here  $u, c$  are consistent with threshold ElGamal while  $\bar{u}$  allows the simulator to answer decryption queries during the security proof and  $e, f$  are part of the zero-knowledge proof of equality of discrete log of  $u$  and  $\bar{u}$ .

During decryption, each party verifies the zero-knowledge proof of knowledge by checking:

$$e = \text{H}(c, u, w, \bar{u}, \bar{w}), \text{ where } w := g^f / u^e, \bar{w} := \bar{g}^f / \bar{u}^e$$

For each party  $i$ , if the check fails, the party outputs  $\perp$ ; otherwise, it outputs  $\hat{m}_i := u^{x_i}$  where  $x_i$  is party  $i$ 's secret key as in the standard ElGamal scheme. A combiner recovers the plaintext in the same manner as the standard ElGamal scheme as described above. Since the zero-knowledge proof is publicly verifiable, the combiner first verifies that the zero-knowledge proof is valid before computing the plaintext.

Similar to threshold ElGamal, the CCA security proof of Shoup-Gennaro relies on embedding the DDH tuple of  $(g^\alpha, g^\beta, g^\zeta)$  into the scheme. The reduction adversary in the Shoup-Gennaro scheme faces the same difficulty as in the standard threshold ElGamal when interacting with an adaptive adversary. Namely, the reduction adversary must provide consistent secret keys upon corruption after committing to each secret key via the per-party public key while the aggregate secret is unknown to the reduction adversary.

We again follow the approach of Das and Ren [DR24] to embed the DDH challenge elsewhere so that the aggregate secret key is known to the reduction adversary. As in our CPA secure scheme, we include an

---

\*We modified TDH2 to rely on the DDH assumption. The original work relies on the Computation Diffie-Hellman assumption.

additional group element,  $h$ , in our public key to help embed the DDH challenge, i.e.,  $h = g^\alpha$  where  $g^\alpha$  is part of the DDH challenge. However, if we modify the public key to  $\text{pk} := g^x h^y$  with  $y = 0$  and rig the public key in our security proof such that  $y \neq 0$  as in our CPA secure scheme, the reduction adversary immediately runs into a problem when simulating decryption queries. As in our CPA secure scheme, the simulated decryption shares are inconsistent with the underlying plaintext with a rigged public key. Recall that under the rigged public key, the decryption protocol will produce  $m' := m \cdot g^{xr} h^{yr} / g^{xr} := m \cdot h^{rv}$  where  $r$  is the chosen randomness in the ciphertext. Thus, the simulation is clearly distinguishable from the real execution of the protocol. The reduction adversary must program  $H(\text{ctxt}) = h^r$  to produce consistent decryption shares of  $\text{ctxt}$ . The CCA adversary, compared to a CPA adversary, can additionally query ciphertexts to obtain all decryption shares of the queried ciphertext. During decryption queries, the reduction adversary runs into two challenges: 1) the adversary knows  $r$ , so  $H(\text{ctxt}) = h^r$  is distinguishable from true randomness; 2)  $r$  is unknown to the reduction adversary.

To remedy the first problem, we introduce a third group element,  $v^z$ , in the public key, i.e.,  $\text{pk} = g^x h^y v^z$  where  $z = 0$ . The inclusion of a third group element allows us to introduce a third programmed random oracle in the partial decryption share to mask  $h^r$  with additional randomness. Then, the partial decryption takes the form of  $\hat{m}_i := u^{x_i} H_2(\text{ctxt})^{y_i} H_3(\text{ctxt})^{z_i}$ . We rig  $z$  such that  $z = 1$  in the security proof.

The simulator programs  $H_2(\text{ctxt}), H_3(\text{ctxt})$  as follows:

$$H_2(\text{ctxt}) := g^b \quad \text{and} \quad H_3(\text{ctxt}) := \frac{h^{ry} v^r}{g^{by}} \quad \text{where } b \leftarrow \$ \mathbb{Z}_p$$

$H_2(\text{ctxt}) = g^b$  is clearly random. Intuitively, the output of  $H_3(\text{ctxt})$  is indistinguishable from true randomness because both  $b$  and  $y$  are unknown to the adversary. By assuming the hardness of DDH, we argue that  $\frac{h^{ry} v^r}{g^{by}}$  is indistinguishable from  $\frac{h^{ry} v^r}{g^\eta}$  where  $\eta$  is sampled randomly. Next, we show that the recovered plaintext is consistent:

$$\hat{m} = u^x \cdot H_2(\text{ctxt})^y \cdot H_3(\text{ctxt})^z = u^x \cdot g^{by} \cdot \frac{h^{ry} v^r}{g^{by}} = u^x h^{ry} v^{rz}$$

The remaining challenge is computing  $H_3(\text{ctxt})$  when  $r$  is unknown to the reduction adversary. More specifically, the reduction adversary needs to compute  $(h^y v)^r$  in order to program  $H_3$ . Recall that the adversary in its decryption query provides both  $u := g^r$  and  $\bar{u} := \bar{g}^r$  where  $\bar{g}$  is a random generator. Borrowing Shoup-Gennaro's proof technique of straightline (without rewinding) extracting  $\text{pk}^r$  from  $\bar{u}$  to simulate decryption queries, we extract  $(h^y v)^r$ . Our simulator picks  $\bar{g} := (h^y v)^k, k \leftarrow \$ \mathbb{Z}_p$ . Then  $(h^y v)^r := \bar{u}^{1/k}$ .

Notice that in the real-world execution of our scheme, when  $y = z = 0$ , the partial decryptions also correctly recover the underlying plaintext  $m$ .

## 2 Preliminaries

**Notations.** For any integer  $a$ , we use  $[a]$  to denote the ordered set  $\{1, 2, \dots, a\}$ . For any set  $S$ , we use  $s \leftarrow \$ S$  to indicate that  $s$  is sampled uniformly randomly from  $S$ . We use  $|S|$  to denote the size of set  $S$ . Throughout the paper, we will use " $\leftarrow$ " for probabilistic assignment and " $:=$ " for deterministic assignment. We use  $\lambda$  to denote the security parameter and  $\text{negl}(\lambda)$  to denote functions negligible in  $\lambda$ . We use the terms *party* (resp. *parties*) and *decryptor* (resp. *decryptors*) interchangeably.

### 2.1 Model

We consider a set of  $n$  parties denoted by  $\{1, 2, \dots, n\}$ . We consider a probabilistic polynomial time (PPT) adversary  $\mathcal{A}$  who can adaptively corrupt up to  $t < n$  out of the  $n$  parties. The adversary can adaptively decide to corrupt a party at any time during the execution of the protocol. Corrupted parties are controlled by the adversary and can deviate arbitrarily from the protocol specification. Note that with  $t \geq n/2$ , i.e., with a dishonest majority, it is impossible to achieve both secrecy and guaranteed output delivery [KL07]. We focus on secrecy over guaranteed output delivery for the dishonest majority case.

## 2.2 Shamir Secret Sharing, NIZK protocols, and Assumptions

**Shamir secret sharing.** The Shamir secret sharing [Sha79] embeds the secret  $s$  in the constant term of a polynomial  $p(x) = s + a_1x + a_2x^2 + \dots + a_dx^d$ , where other coefficients  $a_1, \dots, a_d$  are chosen uniformly randomly from a field  $\mathbb{Z}_p$ . The  $i$ -th share of the secret is  $p(i)$ , i.e., the polynomial evaluated at  $i$ . Given  $d+1$  distinct shares, one can efficiently reconstruct the polynomial and the secret  $s$  using Lagrange interpolation. Also,  $s$  is information-theoretically hidden from an adversary that knows  $d$  or fewer shares.

**NIZK.** Throughout our paper, we will use non-interactive zero-knowledge proofs of knowledge (NIZK) via the Fiat-Shamir heuristic [FS86]. A zero-knowledge proofs of knowledge protocol allows a prover  $\mathcal{P}$  to convince a verifier  $\mathcal{V}$  that he knows a secret witness that satisfies some property without revealing to  $\mathcal{V}$  any additional information about the witness. The starting point of a NIZK protocol via the Fiat-Shamir heuristic is an interactive  $\Sigma$ -protocol, which is a three-move protocol in which  $\mathcal{P}$  first sends  $\mathcal{V}$  a commitment. Then  $\mathcal{V}$  responds with a random challenge. Lastly,  $\mathcal{P}$  responds with a value computed from the random challenge. The Fiat-Shamir heuristic renders the protocol non-interactive by replacing  $\mathcal{V}$ 's random challenge with a hash of all previous messages. We require the NIZK protocols to satisfy the standard *completeness*, *soundness*, and *zero-knowledge*. We adapt the definition of NIZK from [BS23, §20.3].

**Definition 1 (NIZK).** Let  $\mathcal{R}$  be a binary relation and  $(x, y) \in \mathcal{R}$  be a witness statement pair. A NIZK protocol for  $\mathcal{R}$  is a pair of polynomial-time algorithms:

1.  $\text{SigmaProve}(x, y) \rightarrow \pi$ : an algorithm invoked by  $\mathcal{P}$  to generate a proof for  $(x, y) \in \mathcal{R}$ .
2.  $\text{SigmaVer}(y, \pi) \rightarrow 0/1$ : algorithm invoked by  $\mathcal{V}$  to either accept or reject  $\pi$ .

A NIZK protocol satisfies the following properties:

- Completeness. If  $(x, y) \in \mathcal{R}$  and  $\mathcal{P}, \mathcal{V}$  follow the protocol,  $\mathcal{V}$  always outputs ‘accept’.
- Soundness. We say that a NIZK protocol is sound if an adversary can cause  $\mathcal{V}$  to output ‘accept on  $(x, y) \notin \mathcal{R}$  with negligible probability.
- Zero-Knowledge. There exists an efficient probabilistic simulator  $\mathcal{S}$  that takes as input a statement  $y \in \mathcal{R}$  and outputs  $\pi$  that is an accepting proof of  $y$  and has the same distribution as an honestly computed proof for statement  $y$ .

We also use the following two computational assumptions defined for groups generated by the  $\text{GGen}$  algorithm. More precisely, let  $\text{GGen}$  be a group generation algorithm that on input  $1^\lambda$  outputs the description of a prime order group  $\mathbb{G}$ . The description includes the prime order  $p$ , a generator  $g \in \mathbb{G}$ , and a description of the group operation.

**Assumption 1 (DDH)** We say that the decisional Diffie-Hellman (DDH) assumption holds relative to  $\text{GGen}$ , if for all PPT adversaries  $\mathcal{A}$ , the following advantage is negligible:

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \text{GGen}}^{\text{DDH}}(\lambda) := & \left| \Pr \left[ \mathcal{A}(\mathbb{G}, p, g, g^x, g^y, g^{xy}) = 1 \mid \begin{array}{l} (\mathbb{G}, p, g) \leftarrow \text{GGen}(1^\lambda), \\ x, y \leftarrow \mathbb{Z}_p \end{array} \right] \right. \\ & \left. - \Pr \left[ \mathcal{A}(\mathbb{G}, p, g, g^x, g^y, g^z) = 1 \mid \begin{array}{l} (\mathbb{G}, p, g) \leftarrow \text{GGen}(1^\lambda), \\ x, y, z \leftarrow \mathbb{Z}_p \end{array} \right] \right| = \varepsilon_{\text{ddh}}. \end{aligned}$$

**Assumption 2 (DL)** We say that the discrete logarithm (DL) assumption holds relative to  $\text{GGen}$ , if for all PPT adversaries  $\mathcal{A}$ , the following advantage is negligible in  $\lambda$ :

$$\text{Adv}_{\mathcal{A}, \text{GGen}}^{\text{DL}}(\lambda) := \Pr [\mathcal{A}(\mathbb{G}, g, g^\alpha) = \alpha] = \varepsilon_{\text{dl}}.$$

## 2.3 Threshold Decryption

First, we provide a formal definition for a public key threshold decryption scheme.

**Definition 2 (Public Key Threshold Decryption scheme).** Let  $\lambda$  be the security parameter and  $t, n \in \mathbb{N}$  with  $t < n$ . A non-interactive threshold decryption scheme TD for a finite message space  $\mathcal{M}$  is a tuple of polynomial time algorithms  $\text{TD} = (\text{Setup}, \text{KGen}, \text{Enc}, \text{PDec}, \text{PVer}, \text{Comb})$  defined as follows:

1.  $\text{Setup}(1^\lambda, n, t) \rightarrow \text{pp}$  : the setup algorithm takes as input a security parameter and outputs public parameters  $\text{pp}$  (which are given implicitly as input to all other algorithms).
2.  $\text{KGen}(\text{pp}) \rightarrow \text{pk}, \{\text{pk}_i, \text{sk}_i\}_{i \in [n]}$  : the key generation algorithm outputs a public key  $\text{pk}$ , a vector of per party public keys  $\{\text{pk}_1, \dots, \text{pk}_n\}$ , and a vector of secret key shares  $\{\text{sk}_1, \dots, \text{sk}_n\}$ . The  $j$ -th decryptor receives  $(\text{pk}, \{\text{pk}_i\}_{i \in [n]}, \text{sk}_j)$ .
3.  $\text{Enc}(\text{pk}, m) \rightarrow \text{ctxt}$  : the encryption algorithm takes as input a public key  $\text{pk}$  and a message  $m \in \mathcal{M}$ . It outputs a ciphertext  $\text{ctxt}$ .
4.  $\text{PDec}(\text{sk}_i, \text{ctxt}) \rightarrow \hat{m}_i, \pi_i$  : the partial decryption algorithm takes as input a secret key share  $\text{sk}_i$  and a ciphertext  $\text{ctxt}$  and outputs a decryption share  $\hat{m}_i$  for  $c$ , and a proof  $\pi_i$  for  $\hat{m}$ .
5.  $\text{PVer}(\text{pk}_i, \text{ctxt}, \hat{m}_i, \pi_i) \rightarrow 0/1$  : On input the per party public key  $\text{pk}_i$ , the ciphertext  $\text{ctxt}$ , and a decryption share  $\hat{m}_i$ , and a proof for  $\hat{m}_i$ ,  $\pi_i$ . It outputs 0 (reject) or 1 (accept).
6.  $\text{Comb}(\{\text{pk}_i\}_{i \in [n]}, \text{ctxt}, S, \{\hat{m}_i, \pi_i\}_{i \in S}) \rightarrow m$  : the combine algorithm takes as input a set of  $n$  per party public keys  $\{\text{pk}_i\}_{i \in [n]}$ , a ciphertext  $\text{ctxt}$ , a set  $S$  that is a subset of  $\{1, \dots, n\}$  and  $|S| > t$ , and a set of decryption share of  $\text{ctxt}$  and proof pairs,  $\{\hat{m}_i, \pi_i\}_{i \in S}$  and outputs a message  $m$ .

We require a public key threshold decryption scheme to satisfy the correctness, secrecy, and robustness properties. Intuitively, the correctness property guarantees that when all parties behave honestly, the decryption protocol correctly outputs the original message. More formally,

**Definition 3 (Correctness).** Let  $\mathcal{M}$  be the message space. For all  $m \in \mathcal{M}$ , for all obtainable keys,  $\text{pk}, \{\text{pk}_i, \text{sk}_i\} \leftarrow \text{KGen}(\text{pp})$ , for all obtainable ciphertext,  $\text{ctxt} \leftarrow \text{Enc}(m, \text{pk})$ , and any set  $S \subseteq [n], |S| \geq t + 1$ , we have the following:

$$\Pr[\text{Comb}(\{\text{pk}_i\}_{i \in [n]}, \text{ctxt}, S, \{\text{PDec}(\text{sk}_i, \text{ctxt})\}_{i \in S}) \neq m] < \text{negl}(\lambda) \quad (1)$$

Recall that we will provide two schemes, the first achieves CPA security and the second achieves CCA security. We formalize the CPA and CCA security using the  $\text{IND-TCPA}_{\text{TD}}^A$  and  $\text{IND-TCCA}_{\text{TD}}^A$  games in Figure 1, respectively. Below, we provide a brief description of the corresponding security games.

Our indistinguishability under threshold chosen-**plaintext** attack ( $\text{IND-TCPA}$ ) security game  $\text{IND-TCPA}_{\text{TD}}^A$  is based on the exercise 22.8 in [BS23]. Intuitively, in the  $\text{IND-TCPA}_{\text{TD}}^A$  game, an adversary can freely encrypt messages of its choice and see all decryption shares of the chosen messages. The adversary selects two plaintexts and receives the encryption of one of these messages (chosen at random). Additionally, since the adversary is adaptive, it selects up to  $t$  parties at any time to see their secret keys. The encryption scheme is  $\text{IND-TCPA}$  secure if the adversary cannot determine, with probability better than random guessing, which plaintext was encrypted.

Our indistinguishability under threshold chosen-**ciphertext** attack ( $\text{IND-TCCA}$ ) in the presence of an adaptive adversary game  $\text{IND-TCCA}_{\text{TD}}^A$  is identical to the  $\text{IND-TCPA}_{\text{TD}}^A$ , except the adversary additionally gets access to a decryption oracle  $\text{DEC}$ , which it can repeatedly query to decrypt ciphertexts of its choice and see all decryption shares. Note that the adversary cannot query  $\text{DEC}$  with the challenge ciphertext; otherwise, winning the game would be trivial.

**Definition 4 (Indistinguishability under chosen-plaintext attack).** Consider the  $\text{IND-TCPA}_{\text{TD}}^A$  game in Figure 1. We say that a threshold decryption scheme TD is  $\text{IND-TCPA}$  secure if, for all security parameter  $\lambda$ ,  $n, t \in \mathbb{N}$  with  $t < n$ , PPT adversaries  $\mathcal{A}$ , the following guessing advantage is negligible, i.e.,

$$\epsilon_{\text{IND-TCPA}} := \left| \Pr[\text{IND-TCPA}_{\text{TD}}^A(1^\lambda, n, t) \Rightarrow 1] - \frac{1}{2} \right| = \text{negl}(\lambda) \quad (2)$$

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Game <math>\text{IND-TCCA}_{\text{TD}}^{\mathcal{A}}(1^\lambda, n, t)</math>:</p> <ol style="list-style-type: none"> <li>1: <math>\text{pp} \leftarrow \text{Setup}(1^\lambda, n, t)</math> // public parameters</li> <li>2: <math>\mathcal{C} := \emptyset</math></li> <li>3: <math>\text{pk}, \{\text{pk}_i, \text{sk}_i\} \leftarrow \text{KGen}(\text{pp})</math></li> <li>4: <math>\text{inp} := \text{pp}, \text{pk}, \{\text{pk}_i\}_{i \in [n]}</math></li> <li>5: <math>m_0, m_1 \leftarrow \mathcal{A}^{\text{CORR, ENC, DEC}}(\text{inp})</math></li> <li>6: <math>\mu \leftarrow \mathcal{S} \{0, 1\}</math></li> <li>7: <math>\text{ctxt}_\mu \leftarrow \text{Enc}(\text{pk}, m_\mu)</math></li> <li>8: <math>\mu_{\mathcal{A}} \leftarrow \mathcal{A}^{\text{CORR, ENC, DEC}}(\text{ctxt}_\mu)</math></li> <li>9: <b>if</b> <math>\mu = \mu_{\mathcal{A}}</math> <b>then</b></li> <li>10:     <b>return</b> 1</li> <li>11: <b>else</b></li> <li>12:     <b>return</b> 0</li> </ol> <p><b>Oracle</b> <math>\text{CORR}(i)</math>:</p> <ol style="list-style-type: none"> <li>13: <b>if</b> <math> \mathcal{C}  \geq t</math> <b>then</b></li> <li>14:     <b>return</b> <math>\perp</math></li> <li>15: <math>\mathcal{C} := \mathcal{C} \cup \{i\}</math></li> <li>16: <b>return</b> <math>\text{sk}_i</math></li> </ol> <p><b>Oracle</b> <math>\text{ENC}(m)</math>:</p> <ol style="list-style-type: none"> <li>17: <math>\text{ctxt} \leftarrow \text{Enc}(\text{pk}, m)</math></li> <li>18: <math>\hat{m}_i \leftarrow \text{PDec}(\text{sk}_i, \text{ctxt})</math> for all <math>i \in [n]</math></li> <li>19: <b>return</b> <math>(\text{ctxt}, \{\hat{m}_i, \pi_i\}_{i \in [n]})</math></li> </ol> <p><b>Oracle</b> <math>\text{DEC}(\text{ctxt}, i)</math>:</p> <ol style="list-style-type: none"> <li>20: <b>assert</b> <math>\text{ctxt} \neq \text{ctxt}_\mu</math> // <math>\mathcal{A}</math> cannot invoke <math>\text{DEC}</math> on <math>\text{ctxt}_\mu</math></li> <li>21: <math>\hat{m}_i \leftarrow \text{PDec}(\text{sk}_i, \text{ctxt})</math></li> <li>22: <b>return</b> <math>\hat{m}_i</math></li> </ol> | <p>Game <math>\text{RB-TCCA}_{\text{TD}}^{\mathcal{A}}(1^\lambda, n, t)</math>:</p> <ol style="list-style-type: none"> <li>23: <math>\text{pp} \leftarrow \text{Setup}(1^\lambda)</math></li> <li>24: <math>\text{pk}, \{\text{pk}_i, \text{sk}_i\}_{i \in [n]} \leftarrow \text{KGen}(\text{pp})</math></li> <li>25: Let <math>\mathcal{C} := \emptyset, \mathcal{H} := [n]</math></li> <li>26: <math>\text{inp} := \text{pp}, \text{pk}, \{\text{pk}_i\}_{i \in [n]}</math></li> <li>27: <math>\mathcal{S}', \text{ctxt}, \{\hat{m}_i, \pi_i\}_{i \in \mathcal{S}} \leftarrow \mathcal{A}^{\text{CORR, ENC, DEC}}(\text{inp})</math></li> <li>28: <math>T' := \{i \in \mathcal{S}' \mid \text{PVer}(\text{pk}_i, \text{ctxt}, \hat{m}_i, \pi_i) = 1\}</math></li> <li>29: <b>if</b> <math> T'  &lt; t + 1</math> <b>then</b></li> <li>30:     <b>return</b> 0</li> <li>31: <math>\mathcal{S} := [1, \dots, t + 1]</math></li> <li>32: <math>\mathcal{D} := \{\text{PDec}(\text{sk}_i, \text{ctxt})\}_{i \in \mathcal{S}}</math></li> <li>33: <math>m \leftarrow \text{Comb}(\{\text{pk}_i\}_{i \in [n]}, \text{ctxt}, \mathcal{S}, \mathcal{D})</math></li> <li>34: <math>m' \leftarrow \text{Comb}(\{\text{pk}_i\}_{i \in [n]}, \text{ctxt}, T', \{\hat{m}_i, \pi_i\}_{i \in T'})</math></li> <li>35: <b>if</b> <math>m' \neq m</math> <b>then</b></li> <li>36:     <b>return</b> 1</li> <li>37: <b>else</b></li> <li>38:     <b>return</b> 0</li> </ol> <p>Game <math>\text{IND-TCPA}_{\text{TD}}^{\mathcal{A}}(1^\lambda, n, t)</math> and <math>\text{RB-TCPA}_{\text{TD}}^{\mathcal{A}}(1^\lambda, n, t)</math>:</p> <p>Same as <math>\text{IND-TCCA}_{\text{TD}}^{\mathcal{A}}</math> and <math>\text{RB-TCCA}_{\text{TD}}^{\mathcal{A}}</math>, except that <math>\mathcal{A}</math> does not get access to the decryption oracle <math>\text{DEC}</math></p> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Fig. 1: The chosen ciphertext game  $\text{IND-TCCA}_{\text{TD}}^{\mathcal{A}}$  and chosen plaintext game  $\text{IND-TCPA}_{\text{TD}}^{\mathcal{A}}$  and the robustness security game  $\text{RB-TCCA}_{\text{TD}}^{\mathcal{A}}$   $\text{RB-TCPA}_{\text{TD}}^{\mathcal{A}}$  for a non-interactive threshold decryption  $\text{TD} = (\text{Setup}, \text{KGen}, \text{PDec}, \text{Comb})$  with an adaptive adversary  $\mathcal{A}$ .

**Definition 5 (Indistinguishability under Chosen-Ciphertext Attack).** Consider the  $\text{IND-TCCA}_{\text{TD}}^{\mathcal{A}}$  game in Figure 1. We say that a threshold decryption scheme  $\text{TD}$  is  $\text{IND-TCCA}$  secure if, for all security parameters  $\lambda, n, t \in \mathbb{N}$  with  $t < n$ , PPT adversaries  $\mathcal{A}$ , the following guessing advantage is negligible, i.e.,

$$\varepsilon_{\text{IND-TCCA}} := \left| \Pr[\text{IND-TCCA}_{\text{TD}}^{\mathcal{A}}(1^\lambda, n, t) \Rightarrow 1] - \frac{1}{2} \right| = \text{negl}(\lambda) \quad (3)$$

Finally, we require a threshold decryption scheme to satisfy the robustness property [BBH06], either  $\text{RB-TCPA}_{\text{TD}}^{\mathcal{A}}$  or  $\text{RB-TCCA}_{\text{TD}}^{\mathcal{A}}$ , against up to  $t$  actively malicious parties. We formally define the robustness property for our  $\text{IND-CPA}$  secure (resp.  $\text{IND-CCA}$ ) protocol using the  $\text{RB-TCPA}_{\text{TD}}^{\mathcal{A}}$  (resp.  $\text{RB-TCCA}_{\text{TD}}^{\mathcal{A}}$ ) game defined in Figure 1.

Intuitively, the  $\text{RB-TCPA}_{\text{TD}}^{\mathcal{A}}$  and  $\text{RB-TCCA}_{\text{TD}}^{\mathcal{A}}$  games state that an adversary that can corrupt up to  $t$  parties adaptively cannot, with non-negligible probability, produce a ciphertext and a set of decryption shares of that ciphertext such that all decryption shares are valid but do not combine to the actual plaintext that corresponds to the ciphertext.

More formally, we have the following two definitions:

**Definition 6 (TCPA Robustness).** Consider the  $\text{RB-TCPA}_{\text{TD}}^{\mathcal{A}}$  game in Figure 1. We say that a threshold decryption scheme  $\text{TD}$  is robust if, for all security parameters  $\lambda, n, t \in \mathbb{N}$  with  $t < n$ , PPT adversaries  $\mathcal{A}$ , the following advantage is negligible, i.e.,

$$\text{Adv}_{\mathcal{A}, \text{TD}}^{\text{RB-TCPA}}(\lambda) := \Pr[\text{RB-TCPA}_{\text{TD}}^{\mathcal{A}}(1^\lambda, n, t) \Rightarrow 1] = \text{negl}(\lambda) \quad (4)$$

**Definition 7 (TCCA Robustness).** Consider the  $\text{RB-TCCA}_{\text{TD}}^A$  game in Figure 1. We say that a threshold decryption scheme TD is robust if, for all security parameters  $\lambda, n, t \in \mathbb{N}$  with  $t < n$ , PPT adversaries  $\mathcal{A}$ , the following advantage is negligible, i.e.,

$$\text{Adv}_{\mathcal{A}, \text{TD}}^{\text{RB-TCCA}}(\lambda) := \Pr[\text{RB-TCCA}_{\text{TD}}^A(1^\lambda, n, t) \Rightarrow 1] = \text{negl}(\lambda) \quad (5)$$

## 2.4 Helper Lemmas

In this section, we provide helper lemmas for the analysis of our IND-CPA and IND-CCA schemes. We first restate a helper lemma from Naor-Reingold [NR04, Lemma 4.4] that we use in our security proof. We refer the reader to [NR04] for its proof.

**Lemma 1 (Lemma 4.4 in [NR04]).** For any security parameter  $\lambda$ , let  $(\mathbb{G}, p, g) \leftarrow \text{GGen}(1^\lambda)$ . For all  $q \leq \text{poly}(\lambda)$ , assuming hardness of decisional Diffie-Hellman (DDH) assumption in  $\mathbb{G}$ , the following two distributions are indistinguishable.

$$\mathcal{D}_0 := g, g^\alpha, \{(g^{\beta_i}, g^{\gamma_i})\}_{i \in [q_H]} \text{ for } \alpha \leftarrow \mathbb{Z}_p \text{ and } \forall i \in [q] \ (\beta_i, \gamma_i) \leftarrow \mathbb{Z}_p^2 \quad (6)$$

$$\mathcal{D}_1 := g, g^\alpha, \{(g^{\beta_i}, g^{\alpha \cdot \beta_i})\}_{i \in [q]} \text{ for } \alpha \leftarrow \mathbb{Z}_p \text{ and } \forall i \in [q] \ \beta_i \leftarrow \mathbb{Z}_p \quad (7)$$

More precisely, if an adversary  $\mathcal{A}$  can distinguish between a sample from  $\mathcal{D}_0$  and  $\mathcal{D}_1$  with probability  $\varepsilon$ , then  $\mathcal{A}$  can break the DDH assumption with probability at least  $\varepsilon - 1/p$ . This implies  $\varepsilon \leq \varepsilon_{\text{ddh}} + 1/p$ .

Additionally, we restate a helper lemma from [DR24, Lemma 4]. We refer to [DR24] for its proof.

**Lemma 2.** Let  $(X_0, X_1)$  and  $(Y_0, Y_1)$  be two tuples of discrete random variables where  $X_0$  is independent of  $Y_0$ , and  $X_1$  is independent of  $Y_1$ . For every function  $f(X_\theta, Y_\theta)$  for either  $\theta \in \{0, 1\}$ , if  $\Pr[X_0 = x] = \Pr[X_1 = x]$  holds for all  $x \in X_0 \cup X_1$  and  $\Pr[Y_0 = y] = \Pr[Y_1 = y]$  holds for all  $y \in Y_0 \cup Y_1$ , then  $\Pr[f(X_0, Y_0) = z] = \Pr[f(X_1, Y_1) = z]$  holds for all  $z$  in the range of  $f$ .

We define another helper lemma that we adapt from Pedersen's VSS [Ped91].

**Lemma 3 (Adaptive Secrecy of Pedersen Secret Sharing).** Let  $(\mathbb{G}, p, g) \leftarrow \text{GGen}(1^\lambda)$  and let  $h \in \mathbb{G}$  be another uniformly random independent generator of  $\mathbb{G}$ . Let  $y \in \mathbb{Z}_p$  be the secret to be shared and  $y(\cdot)$  be a random polynomial of degree  $t$  such that  $y(0) = y$ . Let  $x \leftarrow \mathbb{Z}_p$  and  $x(\cdot)$  be a random polynomial of degree  $t$  such that  $x(0) = x$ . Let

$$\text{view}_{\mathcal{C}} = \{g, h, g^x h^y, \{g^{x(i)} h^{y(i)}\}_{i \in [n]}, \{x(i), y(i)\}_{i \in \mathcal{C}}\} \quad (8)$$

For all subset  $\mathcal{C}$  of  $\{1, \dots, n\}$  of size at most  $t$  and all  $y^* \in \mathbb{Z}_p$ :

$$\Pr[y = y^* | \text{view}_{\mathcal{C}}] = \Pr[y = y^*] \quad (9)$$

The view defined in lemma 3 is the view of an adversary in the Pedersen VSS scheme. One crucial step in our IND-TCPA proof is showing that an adversary cannot distinguish between an honest pk and a “rigged” pk with a non-zero  $y$ . This relies on the information-theoretic hiding property of Pedersen VSS. In other words, if an adversary cannot distinguish between an execution of Pedersen secret sharing for 0 and one for a non-zero value, then  $\mathcal{A}$  cannot detect the rigged public key. In our IND-TCPA analysis in section 2.3, we rely on this lemma to show that the simulated game with a “rigged” pk is indistinguishable from the real execution of our protocol.

*Proof.* See Theorem 4.4 in [Ped91]. There exists a bijection between our  $\text{view}_{\mathcal{C}}$  defined in eq. (8) and  $\text{view}_{\mathcal{C}}$  defined in [Ped91].

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><u>Setup</u>(<math>1^\lambda, n, t</math>):</p> <ol style="list-style-type: none"> <li>1: <math>(\mathbb{G}, p, g) \leftarrow \text{GGen}(1^\lambda)</math>.</li> <li>2: Let <math>h \in \mathbb{G}</math> be a uniformly random generator of <math>\mathbb{G}</math>.</li> <li>3: Let <math>H : \mathcal{M} \rightarrow \mathbb{G}</math> and <math>H_{FS} : \{0, 1\}^* \rightarrow \mathbb{Z}_p</math> be two hash functions modeled as random oracle.</li> <li>4: <b>return</b> <math>\text{pp} := (\mathbb{G}, p, n, t, g, h, H, H_{FS})</math>.</li> </ol> <p>// We assume all algorithms implicitly take <math>\text{pp}</math> as input.<br/>We use <math>H_{FS}</math> in <math>\text{SigmaProve}</math> and <math>\text{SigmaVer}</math>.</p> <p><u>KGen</u>(<math>\text{pp}</math>):</p> <ol style="list-style-type: none"> <li>5: Let <math>x(\cdot), y(\cdot) \leftarrow \mathbb{Z}_p[x]</math> be two polynomials of degree <math>t</math> with <math>y(0) = 0</math>.</li> <li>6: <math>\text{pk} := g^{x(0)} h^{y(0)} = g^{x(0)}</math></li> <li>7: <b>for</b> each <math>i \in [n]</math> <b>do</b></li> <li>8:   <math>\text{sk}_i := (x(i), y(i))</math></li> <li>9:   <math>\text{pk}_i := g^{x(i)} h^{y(i)}</math></li> <li>10: <b>return</b> <math>(\text{pk}, \{\text{pk}_i\}_{i \in [n]}, \text{sk}_j)</math> to party <math>j</math> for all <math>j \in [n]</math></li> </ol> <p><u>Enc</u>(<math>\text{pk}, m</math>):</p> <ol style="list-style-type: none"> <li>11: <math>r \leftarrow \mathbb{Z}_p</math></li> <li>12: <math>\text{ctxt} := (c, u) = (m \cdot \text{pk}^r, g^r)</math></li> <li>13: <b>return</b> <math>\text{ctxt}</math></li> </ol> | <p><u>PDec</u>(<math>\text{sk}_i, \text{ctxt} = (c, u)</math>):</p> <ol style="list-style-type: none"> <li>14: <math>\hat{m}_i := u^{x(i)} \cdot H(\text{ctxt})^{y(i)}</math></li> <li>15: <math>\pi_i := \text{SigmaProve}(\text{pk}_i, u, \hat{m}_i, \text{sk}_i)</math></li> <li>16: <b>return</b> <math>(\hat{m}_i, \pi_i)</math></li> </ol> <p><u>PVer</u>(<math>\text{pk}_i, \text{ctxt} = (c, u), \hat{m}_i, \pi_i</math>):</p> <ol style="list-style-type: none"> <li>17: <b>return</b> <math>\text{SigmaVer}(\text{pk}_i, u, \hat{m}_i, \pi_i)</math></li> </ol> <p><u>Comb</u>(<math>\{\text{pk}_i\}_{i \in [n]}, \text{ctxt}, S, \{(\hat{m}_i, \pi_i)\}_{i \in S}</math>):</p> <ol style="list-style-type: none"> <li>18: <b>parse</b> <math>(u, c) := \text{ctxt}</math></li> <li>19: <math>T := \emptyset</math></li> <li>20: <b>for</b> each <math>i \in S</math> <b>do</b></li> <li>21:   <b>if</b> <math>\text{PVer}(u, \text{pk}_i, \hat{m}_i, \pi_i)</math> <b>then</b></li> <li>22:     <math>T \cup \{i\}</math></li> <li>23: <b>assert</b> <math> T  \geq t + 1</math></li> <li>24: Let <math>L_{i,T}</math> be the <math>i</math>-th Lagrange coefficients for <math>T</math></li> <li>25: <math>\hat{m} := \prod_{i \in T} \hat{m}_i^{L_{i,T}}</math></li> <li>26: <b>return</b> <math>m := c \cdot \hat{m}^{-1}</math></li> </ol> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Fig. 2: Adaptively IND-CPA secure threshold ElGamal decryption scheme with trusted key generation.

### 3 Design of the IND-TCPA Threshold Decryption Scheme

We now describe our non-interactive IND-TCPA secure threshold decryption scheme based on the ElGamal encryption scheme in Figure 2. Let  $\mathcal{M}$  be the message space.

Setup( $1^\lambda, n, t$ ): Let  $(\mathbb{G}, p, g) \leftarrow \text{GGen}(1^\lambda)$ . Let  $h \in \mathbb{G}$  be an additional uniformly random independent generator of  $\mathbb{G}$ . Let  $H : \mathcal{M} \rightarrow \mathbb{G}$  and  $H_{FS} : \{0, 1\}^* \rightarrow \mathbb{G}$  be two distinct hash functions modeled as random oracles. The public parameters of our scheme are then  $(\mathbb{G}, g, h, H, H_{FS})$ . We assume that all the algorithms below implicitly take the public parameters as input.

KGen( $\text{pp}$ ): The KGen algorithm samples two uniformly random polynomials  $x(\cdot), y(\cdot)$  of degree  $t$  with the constraint that  $y(0) := 0$ . The decryption key of party  $i$  is then  $\text{sk}_i := (x(i), y(i))$ . Let  $\text{pk} := g^{x(0)}$  be the public key, and  $\text{pk}_i := g^{x(i)} h^{y(i)}$  for all  $i \in [n]$  be the per-party public key.

Enc( $\text{pk}, m$ ): Our encryption algorithm is identical to the standard ElGamal encryption scheme. First, generate a random element  $r \leftarrow \mathbb{Z}_p$ , and then the encryption of message  $m$  is the tuple  $\text{ctxt} := (c, u)$  where  $c := m \cdot \text{pk}^r$  and  $u := g^r$ .

PDec( $\text{sk}_i, \text{ctxt} := (c, u)$ ): On input the partial decryption key  $\text{sk}_i$  and the ciphertext tuple  $\text{ctxt} := (c, u)$ , compute partial decryption  $\hat{m}_i := u^{x(i)} \cdot H(\text{ctxt})^{y(i)}$  and a NIZK proof  $\pi_i$  that  $\hat{m}_i$  is correctly computed from the ciphertext.  $\pi_i$  is computed using the NIZK protocol described in Figure 3. The multiplication by  $H(\text{ctxt})^{y(i)}$  is the main difference compared to the static threshold ElGamal encryption.

PVer( $\text{pk}_i, \text{ctxt} := (c, u), \hat{m}_i, \pi_i$ ): On input the per-party public key  $\text{pk}_i$  the ciphertext  $\text{ctxt}$ , and partial decryption share tuple  $(\hat{m}_i, \pi_i)$ , validate the partial decryption share by running the NIZK protocol  $\text{SigmaVer}$  protocol. Output 1 if and only if  $\text{SigmaVer}$  outputs 1.

Comb( $\{\text{pk}_i\}_{i \in [n]}, \text{ctxt} := (c, u), S, \{(\hat{m}_i, \pi_i)\}_{i \in S}$ ): The input is a set of per-party public keys of all parties, the ciphertext  $\text{ctxt} := (c, u)$ , a set of parties  $S$ , a set of partial decryption tuples that correspond to the parties in  $S$ ,  $(\hat{m}_i, \pi_i)$ . The aggregator first validates each decryption tuple using  $\text{PVer}$ . Let  $T$  be the set of indices of

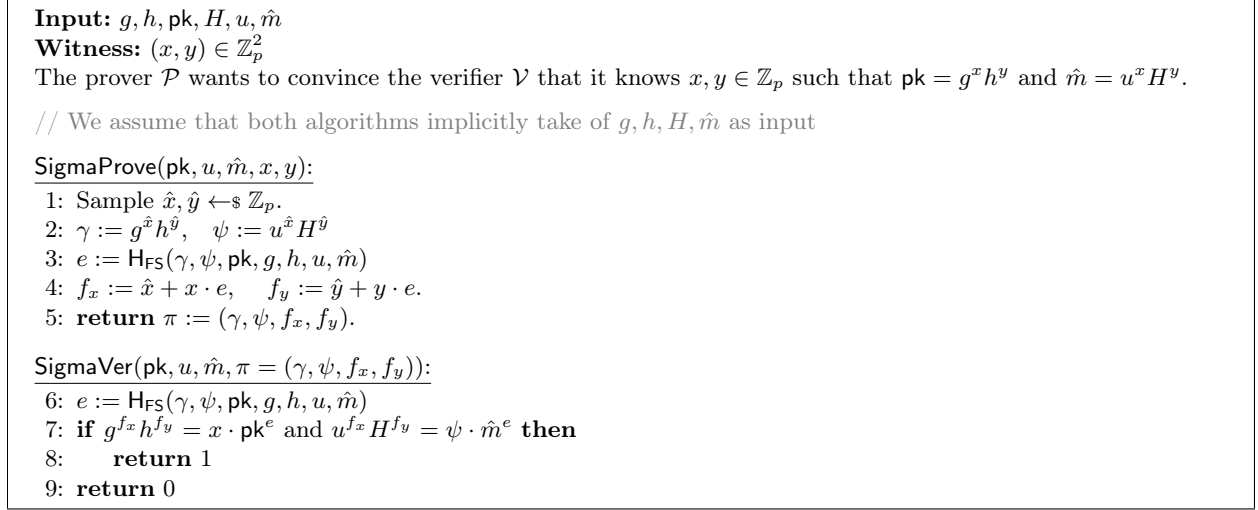


Fig. 3: NIZK-protocol for computing and verifying the correctness proof for partial decryptions.

parties with valid partial decryptions. It then computes the plaintext  $m$  as:

$$m := c \cdot \hat{m}^{-1}, \quad \text{where} \quad \hat{m} := \prod_{i \in T} \hat{m}_i^{L_{i,T}} \quad (10)$$

where  $L_{i,T}$  is the  $i$ -th Lagrange coefficient with respect to the set  $T$ .

## 4 Security Analysis for the IND-TCPA Threshold Decryption Scheme

### 4.1 NIZK Protocol

**Lemma 4.** *The proposed NIZK protocol in Figure 3 satisfies completeness, soundness, and zero knowledge.*

*Proof.* Completeness is straightforward. We now focus on soundness and zero-knowledge.

#### Soundness.

We show that our NIZK protocol satisfies soundness by showing that its underlying  $\Sigma$ -protocol satisfies *knowledge soundness*, i.e., there exists an efficient witness extractor  $\mathcal{E}$  that interacts with a  $\mathcal{P}$  (that produces an accepting proof) and extracts  $x$  such that  $(x, y) \in \mathcal{R}$ .  $\mathcal{E}$  interacts with the prover twice, using two different challenges  $e$  and  $e'$  on the same input. The prover returns two accepting proofs,  $(\gamma, \psi, f_x, f_y), (\gamma, \psi, f'_x, f'_y)$ , where  $e \neq e'$ . The witness  $x = \frac{f_x - f'_x}{e - e'}$  and  $y = \frac{f_y - f'_y}{e - e'}$  can be extracted from the two proofs. Since knowledge soundness implies soundness [Lin01, §3.2], our NIZK protocol satisfies soundness.

Zero-knowledge. We prove zero-knowledge by constructing a simulator  $\mathcal{S}$  such that  $\mathcal{A}$ 's view in its interaction with  $\mathcal{S}$  is distinguishable from its view in the real execution of the protocol with negligible probability  $\varepsilon_{\text{ZK}}$ . Additionally,  $\mathcal{A}$  at most  $q_{\text{FS}}$  queries to  $\text{H}_{\text{FS}}$ . To answer  $\mathcal{A}$ 's random oracle queries,  $\mathcal{S}$  samples responses uniformly at random. To simulate a NIZK proof,  $\mathcal{S}$  samples  $(e, f_x, f_y) \leftarrow \mathbb{Z}_p^3$ , and computes  $(\gamma, \psi)$  as,

$$\gamma = g^{f_x} h^{f_y} \text{pk}^{-e}, \quad \psi = u^{f_x} H^{f_y} \hat{m}^{-e} \quad (11)$$

Finally,  $\mathcal{S}$  programs the random oracle  $\text{H}_{\text{FS}}$  such that  $\text{H}_{\text{FS}}(\gamma, \psi, \text{pk}, g, h, u, \hat{m}) := e$ , and outputs  $(\gamma, \psi, f_x, f_y)$ , which is an accepting transcript with the correct distribution, provided the bad event  $\text{Coll}$  does not occur.

Coll occurs if  $\mathcal{A}$  queries  $H_{FS}$  on  $(\gamma, \psi, \mathbf{pk}, g, h, u, \hat{m})$  before  $\mathcal{S}$  outputs  $(\gamma, \psi, f_x, f_y)$  as the simulated proof. If Coll occurs,  $\mathcal{S}$  cannot program  $H_{FS}$  to the desired  $e$ , causing the simulation to fail.

The simulation fails with the same probability that event Coll occurs. Lastly, we analyze the probability of event Coll. Since  $\gamma, \psi \leftarrow \mathbb{G}$  is sampled uniformly at random, Coll occurs with probability at most  $q_{FS}/p^3$ . Thus  $\varepsilon_{ZK} \leq q_{FS}/p^3$ , which is negligible.

## 4.2 Correctness and Robustness

Our IND-CPA secure threshold decryption scheme satisfies the correctness property and the robustness property defined in definition 3 and definition 6, respectively. We use the NIZK protocol defined in ?? to ensure these two properties of our scheme. Recall that in our scheme, during PDec of ciphertext  $\text{ctxt}$ , each party  $i$  acts as the prover and computes a NIZK proof  $\pi_i$  for the correctness of the partial decryption  $\hat{m}_i$  in relation to the per-party public key  $\mathbf{pk}_i$  as part of the decryption share. More specifically, each party  $i$  computes  $\pi_i$  for the following statement: party  $i$  knows secret key  $(x_i, y_i)$  such that  $\mathbf{pk}_i = g^{x_i} h^{y_i}$  and  $\hat{m}_i = u^{x_i} H(\text{ctxt})^{y_i}$ .

The combiner, upon receiving a partial decryption in the form of  $(\hat{m}_i, \pi_i)$ , acts as the verifier in definition 1 and verifies  $\pi_i$ . Only  $\hat{m}_i$  which corresponding  $\pi_i$  is accepted (verified to true) is used in recovering the plaintext.

The correctness of our scheme is due to the completeness of the NIZK protocol since all honestly computed  $(\hat{m}_i, \pi_i)$  pairs will be accepted by the combiner.

The robustness of our scheme follows from the soundness property of the NIZK protocol. Intuitively, the soundness property of the NIZK protocol ensures that dishonestly computed decryption shares do not contribute to the recovery of the plaintext with high probability. More formally, we have the following lemma:

**Lemma 5 (TCPA-Robustness).** *Assuming the NIZK protocol satisfies soundness defined in definition 1, the threshold decryption in fig. 2 is RB-TCPA<sub>TD</sub><sup>A</sup> secure.*

*Proof.* Let  $\mathcal{B}$  denote an adversary against the soundness game defined in definition 1. Let  $\mathcal{A}$  be an adversary against the RB-TCPA<sub>TD</sub><sup>A</sup> game in fig. 1.  $\mathcal{B}$  interacts with  $\mathcal{A}$  by honestly providing  $\mathbf{pk}$  and  $\{\mathbf{pk}_i\}_{i \in [n]}$  by sampling secret polynomials  $x(\cdot), y(\cdot)$ .  $\mathcal{B}$  also honestly answers all oracle queries according to the protocol specification. Observe that  $\mathcal{A}$  wins the RB-TCPA<sub>TD</sub><sup>A</sup> game in two ways. The first way is by violating the correctness property, defined in definition 3, of our scheme. More precisely,  $\mathcal{A}$  wins the RB-TCPA<sub>TD</sub><sup>A</sup> game by finding a set  $\mathcal{S}'$  of parties whose honestly computed partial decryptions combine to a plaintext that differs from the one produced by set  $\mathcal{S}$ . **Ziling: see change** The second way is by forging at least one accepted  $\pi_i$  for a dishonestly computed  $\hat{m}_i$ . Thus,  $\mathcal{B}$  wins the soundness game by identifying and outputting such an accepted but invalid  $\pi_i$ , statement pair. Since  $\mathcal{B}$  knows the witness, i.e., the secret key of party  $i$ , it can verify that  $\pi_i$  proves an incorrect statement.  $\square$

## 4.3 IND-TCPA against an Adaptive Adversary

We will prove IND-TCPA assuming the decisional Diffie-Hellman (DDH) assumption in  $\mathbb{G}$  via a sequence of games. Game  $\mathbf{G}_0$  is the IND-TCPA<sub>TD</sub><sup>A</sup> game in which  $\mathcal{A}$  makes queries to the encryption oracle and tries to guess the challenge bit  $\mu$ . In game  $\mathbf{G}_5$ ,  $\mathcal{A}$  receives a ciphertext  $\text{ctxt}$  that is independent of the challenge bit  $\mu$ . Therefore, the probability that  $\mathcal{A}$  wins game  $\mathbf{G}_5$  is exactly  $1/2$ . We use a sequence of games to bound the difference between  $\mathcal{A}$ 's advantage in game  $\mathbf{G}_0$  and game  $\mathbf{G}_5$ .

For any adversary  $\mathcal{A}$  for the  $k$ -th game  $\mathbf{G}_k$ , we will use  $G_k$  to denote the event that the game outputs 1, i.e.,  $\mathcal{A}$  wins the game  $\mathbf{G}_k$ .

**GAME  $\mathbf{G}_0$ :** This is the security game IND-TCPA<sub>TD</sub><sup>A</sup> for our threshold decryption scheme, where the game follows the honest protocol. The game provides  $\mathcal{A}$  access to random oracles,  $H$  and  $H_{FS}$ , using the standard lazy simulation technique, i.e., the game uniformly and independently samples outputs of the random oracle on demand when  $\mathcal{A}$  first queries the random oracle on a specific input.

We assume that  $\mathcal{A}$  always corrupts exactly  $t$  parties. This assumption does not change  $\mathcal{A}$ 's winning advantage. This is without generality: we can build a wrapper adversary that invokes  $\mathcal{A}$  and then additionally corrupt parties such that exactly  $t$  parties are corrupted.

**GAME  $\mathbf{G}_1$ :** This game is identical to  $\mathbf{G}_0$ , except that we sample  $\alpha_h \leftarrow \mathbb{Z}_p^*$  and set  $h := g^{\alpha_h}$ . Clearly,  $\Pr[G_0] = \Pr[G_1]$ .

**GAME  $\mathbf{G}_2$ :** This game is identical to  $\mathbf{G}_1$ , except for each encryption query we program the random oracle so that for the  $j$ -th encryption query on message  $m_j$ , we set  $H(\text{ctxt}_j) := h^{r_j}$ , where  $\text{ctxt}_j := (m_j \cdot g^{r_j}, g^{r_j})$  for  $r_j \leftarrow \mathbb{Z}_p$ .

The indistinguishability between  $\mathcal{A}$ 's view in game  $\mathbf{G}_1$  and  $\mathbf{G}_2$  is a crucial step of our proof. We prove this indistinguishability assuming the hardness of DDH in  $\mathbb{G}$ . More formally,

**Lemma 6.** *Let  $\varepsilon_{\text{ddh}}$  be the advantage of breaking DDH in  $\mathbb{G}$  as defined in Assumption 1, then  $|\Pr[G_1] - \Pr[G_2]| \leq \varepsilon_{\text{ddh}} + 1/p$ .*

*Proof.* Recall the distributions  $\mathcal{D}_0$  and  $\mathcal{D}_1$  defined in lemma 1. The only difference between the two games is that  $(h, \{u_j, H(\text{ctxt}_j)\}_{j \in [q_h]})$  in game  $\mathbf{G}_1$  is sampled from  $\mathcal{D}_0$  and in game  $\mathbf{G}_2$  is sampled from  $\mathcal{D}_1$ . The rest of the game is identical. Therefore, by lemma 1,  $|\Pr[G_1] - \Pr[G_2]| \leq \varepsilon_{\text{ddh}} + 1/p$ .

**GAME  $\mathbf{G}_3$ :** This game is identical to  $\mathbf{G}_2$ , except that we use simulated proofs for correctness of partial decryption instead of actual NIZK proofs. Looking ahead, we switch to simulated NIZK proofs in this game to later argue in game  $\mathbf{G}_5$  that the NIZK proofs do not reveal any information about the secret keys. This is crucial to argue the indistinguishability between game  $\mathbf{G}_4$  and  $\mathbf{G}_5$ . We have the following lemma.

**Lemma 7.** *Let  $q_e$  be the upper bound on the number of encryption queries,  $\varepsilon_{\text{zk}}$  be the distinguishing advantage of a simulated NIZK proof, then,*

$$|\Pr[G_3] - \Pr[G_2]| \leq q_e \cdot n \cdot \varepsilon_{\text{zk}} \quad (12)$$

*Proof.* The NIZK protocol (Figure 3) we use is Zero-Knowledge. Hence, conditioned on the successful simulation of the NIZK proofs, the views of the  $\mathcal{A}$  in games  $\mathbf{G}_2$  and  $\mathbf{G}_3$  are identically distributed. Since  $\mathcal{A}$  makes at most  $q_e$  encryption queries, and we need to simulate at most  $n$  partial decryptions per encryption query, the lemma follows from a simple union bound.

**GAME  $\mathbf{G}_4$ :**

This game is identical to  $\mathbf{G}_3$  except we sample the secret key polynomial  $v(\cdot)$  such that  $v(0) \leftarrow \mathbb{Z}_p^*$ . This is another crucial step in our proof where we introduce a rigged  $\text{pk}$

**Lemma 8.**  $\Pr[G_3] = \Pr[G_4]$

*Proof.* To help us prove this lemma, we define a distinguishing game  $\mathbf{G}_{\text{vss}}$ . We show that distinguishing between game  $\mathbf{G}_3$  and game  $\mathbf{G}_4$  can be reduced to winning game  $\mathbf{G}_{\text{vss}}$ . In  $\mathbf{G}_{\text{vss}}$ , a distinguisher  $\mathcal{B}$  interacts with a challenger  $\text{Chal}$  that works as follows.

1. Sample  $g, h \leftarrow \mathbb{G}$  and send  $(g, h)$  to  $\mathcal{B}$ .
2. Sample  $x \leftarrow \mathbb{Z}_p$  and a random degree  $t$  polynomial  $x(\cdot)$  such that  $x(0) = x$ .
3. Sample a random bit  $\mu \leftarrow \{0, 1\}$ . If  $\mu = 0$ , let  $y := 0$ , otherwise, let  $y \leftarrow \mathbb{Z}_p$ . Sample degree  $t$  random polynomial  $y(\cdot)$  such that  $y(0) = y$ .
4. Send  $(g^x h^y, \{g^{x(i)} h^{y(i)}\}_{i \in [n]})$  to  $\mathcal{B}$ .
5. Upon receiving an index  $i \in [n]$  from  $\mathcal{B}$  for the  $j$ -th time for  $j \in [t]$ , respond with  $(x(i), y(i))$ .

At the end of the interaction,  $\mathcal{B}$  outputs a bit. Let  $\varepsilon_{\text{vss}}$  denote  $\mathcal{B}$ 's distinguishing advantage, i.e.,  $\varepsilon_{\text{vss}} := |\Pr[\mathcal{B} \Rightarrow 1 | \mu = 1] - \Pr[\mathcal{B} \Rightarrow 1 | \mu = 0]|$ .

Intuitively, in game  $\mathbf{G}_{\text{VSS}}$ ,  $\mathcal{B}$  receives the commitment of some  $x, y$  and the commitments of all Shamir sharings of  $x, y$ . Then  $\mathcal{B}$  adaptively corrupts at most  $t$  shares of  $x, y$ . Lastly,  $\mathcal{B}$  attempts to distinguish if  $y = 0$ .

Next, we show that the difference between games  $G_3$  and  $G_4$  is exactly  $\varepsilon_{\text{VSS}}$  by building a reduction adversary  $\mathcal{B}$  against the  $\mathbf{G}_{\text{VSS}}$  game from  $\mathcal{A}$ .  $\mathcal{B}$  interacts with  $\mathcal{A}$  as follows:

- Upon receiving  $(g^x h^y, \{g^{x(i)} h^{y(i)}\}_{i \in [n]})$ ,  $\mathcal{B}$  sends them to  $\mathcal{A}$  as  $\text{pk}$  and  $\{\text{pk}_i\}_{i \in [n]}$ .
- When  $\mathcal{A}$  queries the encryption oracle on message  $m$ ,  $\mathcal{B}$  responds with  $\text{ctxt} := (m \cdot \text{pk}^r, g^r), \{\hat{m}_i := \text{pk}_i^r\}_{i \in [n]}$  for  $r \leftarrow \mathbb{Z}_p$ .
- when  $\mathcal{A}$  queries the corruption oracle on party  $i$ ,  $\mathcal{B}$  sends  $i$  to  $\text{Chal}$  and receives  $(x(i), y(i))$ .  $\mathcal{B}$  sends  $(x(i), y(i))$  to  $\mathcal{A}$  as party  $i$ 's secret key.
- When  $\mathcal{A}$  challenges  $m_0, m_1$ ,  $\mathcal{B}$  samples random coin  $\mu' \leftarrow \{0, 1\}$ .  $\mathcal{B}$  encrypts  $m_{\mu'}$  as  $\text{ctxt} := (m_{\mu'} \cdot \text{pk}^r, g^r)$  for  $r \leftarrow \mathbb{Z}_p$ .  $\mathcal{B}$  sends  $\text{ctxt}$  to  $\mathcal{A}$  and receives bit  $\mu_{\mathcal{A}} \in \{0, 1\}$ . If  $\mu_{\mathcal{A}} = \mu'$ ,  $\mathcal{B}$  outputs 1.  $\mathcal{B}$  outputs 0 if otherwise.

When  $\mu = 0$ , i.e.,  $y := 0$ ,  $\mathcal{A}$ 's view is identical to that of game  $\mathbf{G}_3$ . Similarly, when  $\mu = 1$ , i.e.,  $y \leftarrow \mathbb{Z}_p$ ,  $\mathcal{A}$ 's view is identical to that of game  $\mathbf{G}_4$ . Then we have  $|\Pr[G_3] - \Pr[G_4]| = |\Pr[\mathcal{B} \Rightarrow 1 | \mu = 0] - \Pr[\mathcal{B} \Rightarrow 1 | \mu = 1]| = \varepsilon_{\text{VSS}}$ .

Lastly, we show that  $\varepsilon_{\text{VSS}} := 0$  and thus  $\Pr[G_3] = \Pr[G_4]$ . To prove this, we rely on the  $H$ -coefficient technique [Pat08, CS14]. Without loss of generality, we can assume that the optimal adversary  $\mathcal{B}$  is unbounded and deterministic.

Let  $W_0$  and  $W_1$  denote the world when  $\mu = 0$  and  $\mu = 1$ , respectively. Let  $T$  be the random variables denoting the transcripts of the interaction of  $\mathcal{D}$  in either  $W_0$  or  $W_1$  for fixed queries from  $\mathcal{B}$ . From Lemma 3,  $\Pr[T = \tau | W_0] = \Pr[T = \tau | W_1]$ . Then by  $H$ -coefficient,  $W_0$  and  $W_1$  are perfectly indistinguishable, i.e.,  $\Pr[\mathcal{B} \Rightarrow 1 | \mu = 1] = \Pr[\mathcal{B} \Rightarrow 1 | \mu = 0]$ .

**GAME  $\mathbf{G}_5$ :** In this game, we change the challenge ciphertext, i.e., instead of sending a valid encryption, we send  $\mathcal{A}$  a random tuple. More precisely,  $\mathcal{A}$ , after challenging  $m_0, m_1$ , receives  $\text{ctxt}_{\mu} := (g^{\beta}, m_{\mu} \cdot (g^{\beta})^x (g^{\zeta})^y)$  where  $\mu \leftarrow \{0, 1\}$ ,  $g^{\beta}, g^{\zeta} \leftarrow \mathbb{G}$ . Recall that in game  $\mathbf{G}_4$ , the challenge ciphertext is of the form  $\text{ctxt}_{\mu} := (g^{\beta}, m_{\mu} \cdot (g^{\beta})^x (g^{\alpha\beta})^y)$  for some random  $\alpha, \beta \leftarrow \mathbb{Z}_p$ . In both games  $\mathbf{G}_4$  and  $\mathbf{G}_5$ ,  $\text{pk} := g^x h^y$ , where  $h := g^{\alpha}$ .

**Lemma 9.**  $|\Pr[G_4] - \Pr[G_5]| \leq \varepsilon_{\text{ddh}}$

*Proof.* We will prove this by building an adversary against DDH assumption,  $\mathcal{A}_{\text{ddh}}$  from  $\mathcal{A}$ .  $\mathcal{A}_{\text{ddh}}$  receives a tuple  $(g^{\alpha}, g^{\beta}, g^{\zeta})$  and interacts with  $\mathcal{A}$  as described. Then if  $\mathcal{A}_{\text{ddh}}$ 's input is a DDH tuple, then  $\mathcal{A}$ 's view is exactly that of game  $\mathbf{G}_6$ . If  $\mathcal{A}_{\text{ddh}}$ 's input is a random tuple, then  $\mathcal{A}$ 's view is exactly that of game  $\mathbf{G}_7$ .

We have the following:

$$\Pr[\mathcal{A}_{\text{ddh}}(g^x, g^y, g^z) = 1 | z = xy] = \Pr[G_4] \quad (13)$$

$$\Pr[\mathcal{A}_{\text{ddh}}(g^x, g^y, g^z) = 1 | z \leftarrow \mathbb{Z}_p] = \Pr[G_5] \quad (14)$$

Hence,  $\mathcal{A}_{\text{ddh}}$ 's advantage is  $|\Pr[G_4] - \Pr[G_5]| \leq \varepsilon_{\text{ddh}}$ .  $\square$

In the last part of our analysis, we will prove that  $\mathcal{A}$  wins in game  $\mathbf{G}_5$  with probability  $1/2$ . Intuitively,  $\mathcal{A}$  receives a challenge ciphertext independent of  $m_0$  and  $m_1$ ; thus, the probability of  $\mathcal{A}$  winning is equivalent to random guessing.

**Lemma 10.**  $\Pr[G_5] = 1/2$ .

*Proof.* The only part in  $\mathcal{A}$ 's view that depends on the hidden bit  $\mu$  is  $\text{ctxt}_{\mu}$ . In game  $\mathbf{G}_5$ ,  $\mathcal{A}$  receives  $\text{ctxt}_{\mu} = (c, u) = (m_{\mu} \cdot g^{\beta x} \cdot g^{\zeta y}, g^{\beta})$  from the game. Since  $\beta, \zeta \leftarrow \mathbb{Z}_p$ ,  $\text{ctxt}_{\mu}$  is uniformly randomly distributed

in  $\mathbb{G}$  for both  $m_0$  and  $m_1$ , independent of  $\mu$ . There is exactly one  $(\beta, \zeta)$  that can result in any feasible  $(c, u)$  for both  $\mu = 0$  and  $\mu = 1$ . Thus, for random variable  $(c, u)$ , all feasible  $(c^*, u^*)$ :

$$\begin{aligned} \Pr[(c, u) = (c^*, u^*) | \mu = 0] &= \Pr[(c, u) = (c^*, u^*) | \mu = 1] \\ &= \Pr[\beta = \beta^*] \cdot \Pr[\zeta = \zeta^*] = \frac{1}{p^2} \end{aligned} \quad (15)$$

Therefore, we conclude that

$$\Pr[\mathcal{A} = 1 | b = 0] = \Pr[\mathcal{A} = 1 | b = 1] = \Pr[G_5] = 1/2. \quad \square$$

We now prove the following main theorem.

**Theorem 3 (Adaptively IND-TCPA Secure Threshold Encryption).** *Let  $\lambda$  be the security parameter and  $(\mathbb{G}, p, g) \leftarrow \text{GGen}(1^\lambda)$  be a group of prime order  $p$ . Let  $n \in \mathbb{N}$  and  $t < n$ . Consider any PPT adversary that makes at most  $q_e$  encryption queries,  $q_h$  random oracle queries to  $\mathbf{H}$ . Let  $\varepsilon_{\text{ZK}}$  be the adversary's advantage in distinguishing simulated NIZK proofs. Then, assuming hardness of DDH in  $\mathbb{G}$  and the random oracle model, the adversary has a guessing advantage in the  $\text{IND-TCPA}_{\text{TD}}^A$  game (Figure 1) for our scheme in Figure 2 at most  $\varepsilon_{\text{IND-TCPA}}$  where*

$$\varepsilon_{\text{IND-TCPA}} \leq 2\varepsilon_{\text{ddh}} + \varepsilon_{\text{ZK}} \cdot q_e \cdot n + \frac{1}{p} \quad (16)$$

where  $\varepsilon_{\text{ddh}}$  is the advantage an adversary in breaking DDH in  $\mathbb{G}$ .

*Proof.* From the above series of games and lemma 16, we get

$$|\Pr[G_0] - \frac{1}{2}| \leq \varepsilon_{\text{IND-TCPA}} \leq 2\varepsilon_{\text{ddh}} + \varepsilon_{\text{ZK}} \cdot q_e \cdot n + \frac{1}{p} \quad (17)$$

which is  $\mathcal{A}$ 's guessing advantage against our threshold encryption scheme.  $\square$

## 5 Design of the IND-TCCA Threshold Decryption Scheme

In this section, we describe our second threshold decryption scheme based on the Shoup-Gennaro scheme, which achieves IND-CCA security. Recall that the Shoup-Gennaro scheme is a variant of the ElGamal encryption scheme with additional ciphertext elements that render it non-malleable, thus achieving CCA security. We now describe our non-interactive IND-CCA secure threshold decryption scheme, and summarize it in Figure 2. Let  $\mathcal{M}$  denote the message space.

**Setup**( $1^\lambda, n, t$ ): Let  $(\mathbb{G}, p, g) \leftarrow \text{GGen}(1^\lambda)$ . Let  $h, v, \bar{g} \in \mathbb{G}$  be additional independent uniformly random generators of  $\mathbb{G}$ . Let  $\mathbf{H}_1 : \{0, 1\}^\ell \times \mathbb{G}^4 \rightarrow \mathbb{Z}_p$ ,  $\mathbf{H}_2, \mathbf{H}_3 : \{0, 1\}^\ell \times \mathbb{G}^2 \times \mathbb{Z}_p^2 \rightarrow \mathbb{G}$  and  $\mathbf{H}_{\text{FS}} : \{0, 1\}^* \rightarrow \mathbb{G}$  be four distinct hash functions modeled as random oracles. The public parameters of our scheme are then  $(\mathbb{G}, g, h, v, \bar{g}, \mathbf{H}_1, \mathbf{H}_2, \mathbf{H}_3, \mathbf{H}_{\text{FS}})$ .

We assume that all the algorithms below implicitly take the public parameters as input.

**KGen(pp)**: The KGen algorithm samples three uniformly random polynomials  $x(\cdot), y(\cdot), z(\cdot)$  of degree  $t$  with the constraint that  $y(0) := 0$  and  $z(0) := 0$ . The decryption key of party  $i$  is then  $\text{sk}_i := (x(i), y(i), z(i))$ . Let  $\text{pk} := g^{x(0)}$  be the public key,  $\text{pk}_i := g^{x(i)} h^{y(i)} v^{z(i)}$  for all  $i \in [n]$  be the per-party public key.

**Enc(pk, m)**: Our encryption algorithm is identical to the Shoup-Gennaro encryption scheme [SG98]. First, sample  $r, s \leftarrow \mathbb{Z}_p$ , and then compute the following:  $c := \text{pk}^r \cdot m$ ,  $u := g^r$ ,  $w := g^s$ ,  $\bar{u} := \bar{g}^r$ ,  $\bar{w} := \bar{g}^s$ ,  $e := \mathbf{H}_1(c, u, w, \bar{u}, \bar{w})$ , and  $f := s + er$ . The ciphertext is  $\text{ctxt} := (c, u, \bar{u}, e, f)$ .

**PDec(sk<sub>i</sub>, ctxt = (c, u,  $\bar{u}$ , e, f))**: On input the partial decryption key  $\text{sk}_i$  and the ciphertext tuple  $\text{ctxt} := (c, u, \bar{u}, e, f)$ , check

$$e = \mathbf{H}_1(c, u, w, \bar{u}, \bar{w}), \text{ where } w := g^f / u^e, \bar{w} := \bar{g}^f / \bar{u}^e \quad (18)$$

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Setup</b> ( $1^\lambda, n, t$ ):<br>1: $(\mathbb{G}, p, g) \leftarrow \text{GGen}(1^\lambda)$ .<br>2: Let $h, v \in \mathbb{G}$ be a uniformly random generator of $\mathbb{G}$ .<br>3: Let $H_1 : \{0, 1\}^l \times \mathbb{G}^4 \rightarrow \mathbb{Z}_p$ , $H_2, H_3 : \{0, 1\}^l \times \mathbb{G}^2 \times \mathbb{Z}_p^2 \rightarrow \mathbb{G}$ and $H_{FS} : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ be four hash functions modeled as random oracle.<br>4: <b>return</b><br>$\text{pp} := (\mathbb{G}, p, n, t, g, h, H_1, H_2, H_3, H_{FS})$ .<br>// We assume all algorithms implicitly take pp as input.<br>We use $H_{FS}$ in SigmaProve and SigmaVer.<br><b>KGen</b> (pp):<br>5: Let $x(\cdot), y(\cdot), z(\cdot) \leftarrow \mathbb{Z}_p[x]$ be three polynomials of degree $t$ with $y(0) := 0, z(0) := 0$ .<br>6: $\text{pk} := g^{x(0)} h^{y(0)} v^{z(0)} = g^{x(0)}$<br>7: <b>for</b> each $i \in [n]$ <b>do</b><br>8: $\text{sk}_i := (x(i), y(i), z(i))$<br>9: $\text{pk}_i := g^{x(i)} h^{y(i)} v^{z(i)}$<br>10: <b>return</b> $(\text{pk}, \{\text{pk}_i\}_{i \in [n]}, \text{sk}_j)$ to party $j$ for all $j \in [n]$<br><b>Enc</b> (pk, m):<br>11: $r, s \leftarrow \mathbb{Z}_p$<br>12: $(c, u) := (m \cdot \text{pk}^r, g^r)$<br>13: $w, \bar{u}, \bar{w} := g^s, \bar{g}^r, \bar{g}^s$<br>14: $e := H_1(c, u, w, \bar{u}, \bar{w})$<br>15: $f := s + er$<br>16: <b>return</b> $\text{ctxt} := (c, u, \bar{u}, e, f)$ | <b>PDec</b> ( $\text{sk}_i, \text{ctxt} = (c, u, \bar{u}, e, f)$ ):<br>17: <b>if</b> $e \neq H_1(c, u, w, \bar{u}, \bar{w})$ where $w := \frac{g^f}{u^e}, \bar{w} := \frac{\bar{g}^f}{\bar{u}^e}$ <b>then</b><br>18: <b>return</b> $\perp$<br>19: $\hat{m}_i := u^{x(i)} H_2(\text{ctxt})^{y(i)} H_3(\text{ctxt})^{z(i)}$<br>20: $\pi_i := \text{SigmaProve}(\text{pk}_i, u, \hat{m}_i, \text{sk}_i)$<br>21: <b>return</b> $(\hat{m}_i, \pi_i)$<br><b>PVer</b> ( $\text{pk}_i, c = (c_1, c_2), \hat{m}_i, \pi_i$ ):<br>22: <b>return</b> $\text{SigmaVer}(\text{pk}_i, u, \hat{m}_i, \pi_i)$<br><b>Comb</b> ( $\{\text{pk}_i\}_{i \in [n]}, \text{ctxt}, S, \{(\hat{m}_i, \pi_i)\}_{i \in S}$ ):<br>23: <b>parse</b> $(c, u, \bar{u}, e, f) := \text{ctxt}$<br>24: $T := \emptyset$<br>25: <b>for</b> each $i \in S$ <b>do</b><br>26: <b>if</b> $\hat{m}_i = \perp$ <b>then</b><br>27: <b>return</b> $\perp$<br>28: <b>if</b> $e \neq H_1(c, u, w, \bar{u}, \bar{w})$ where $w := \frac{g^f}{u^e}, \bar{w} := \frac{\bar{g}^f}{\bar{u}^e}$ <b>then</b><br>29: <b>return</b> $\perp$<br>30: <b>if</b> $\text{PVer}(u, \text{pk}_i, \hat{m}_i, \pi_i)$ <b>then</b><br>31: $T \cup \{i\}$<br>32: <b>assert</b> $ T  \geq t + 1$<br>33: Let $L_{i,T}$ be the $i$ -th Lagrange coefficients for $T$<br>34: $\hat{m} := \prod_{i \in T} \hat{m}_i^{L_{i,T}}$<br>35: <b>return</b> $m := c \cdot \hat{m}^{-1}$ |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Fig. 4: Adaptively IND-CCA secure threshold ElGamal decryption scheme with trusted key generation.

Output  $\perp$  if the check fails. Otherwise, compute partial decryption  $\hat{m}_i := u^{x(i)} \cdot H_2(\text{ctxt})^{y(i)} \cdot H_3(\text{ctxt})^{z(i)}$  and a NIZK proof  $\pi_i$  that  $\hat{m}_i$  is correctly computed from the ciphertext.  $\pi_i$  is computed using  $\Sigma$ -protocol described in Figure 3.

**PVer**( $\text{pk}_i, \text{ctxt} := (c, u, \bar{u}, e, f), \hat{m}_i, \pi_i$ ): On input the threshold public key  $\text{pk}_i$ , the ciphertext  $\text{ctxt} := (c, u, \bar{u}, e, f)$ , and partial decryption share tuple  $(\hat{m}_i, \pi_i)$ , validate the partial decryption share by running the  $\Sigma$ -protocol SigmaVer protocol. Output 1 if and only if SigmaVer outputs 1.

**Comb**( $\{\text{pk}_i\}_{i \in [n]}, \text{ctxt} := (c, u, \bar{u}, e, f), S, \{(\hat{m}_i, \pi_i)\}_{i \in S}$ ): The input is a set of per party public keys of all parties, the ciphertext  $\text{ctxt} := (c, u, \bar{u}, e, f)$ , a set of parties  $S$ , a set of partial decryption tuples that correspond to the parties in  $S$ ,  $(\hat{m}_i, \pi_i)$  for each  $i \in S$ . The combiner first validates the ciphertext by running the check in equation 18 and outputs  $\perp$  if equation 18 evaluates to 0. Then the combiner uses PVer to validate each non- $\perp$  decryption share in  $S$ . Let  $T$  be the set of indices of parties with valid partial decryptions. It then computes the plaintext  $m$  as:

$$m := c \cdot \hat{m}^{-1}, \quad \text{where} \quad \hat{m} := \prod_{i \in T} \hat{m}_i^{L_{i,T}} \quad (19)$$

where  $L_{i,T}$  is the  $i$ -th Lagrange coefficient with respect to the set  $T$ .

## 6 Security Analysis for the IND-TCCA Threshold Decryption Scheme

Our NIZK protocol described in Figure 5 satisfies completeness, soundness, and zero knowledge. We omit analysis of our NIZK protocol and refer the readers to Lemma 4.

**Input:**  $g, h, v, \text{pk}, H_2, H_3, u, \hat{m}$   
**Witness:**  $(x, y, z) \in \mathbb{Z}_p^3$   
The prover  $\mathcal{P}$  wants to convince the verifier  $\mathcal{V}$  that it knows  $x, y, z \in \mathbb{Z}_p$  such that  $\text{pk} = g^x h^y v^z$  and  $\hat{m} = u^x H_2^y H_3^z$ .

// We assume that both algorithms implicitly take of  $g, h, v, u, H_2, H_3, \hat{m}$  as input

SigmaProve(pk, u,  $\hat{m}$ , x, y, z):  
1: Sample  $\hat{x}, \hat{y}, \hat{z} \leftarrow \mathbb{Z}_p$ .  
2:  $\gamma := g^{\hat{x}} h^{\hat{y}} v^{\hat{z}}, \quad \psi := u^{\hat{x}} H_2^{\hat{y}} H_3^{\hat{z}}$   
3:  $e := \text{HFS}(\gamma, \psi, \text{pk}, u, \hat{m})$   
4:  $f_x := \hat{x} + x \cdot e, \quad f_y := \hat{y} + y \cdot e, \quad f_z := \hat{z} + z \cdot e$ .  
5: **return**  $\pi := (\gamma, \psi, f_x, f_y, f_z)$ .

SigmaVer(pk, u,  $\hat{m}$ ,  $\pi = (\gamma, \psi, f_x, f_y, f_z)$ ):  
6:  $e := \text{HFS}(\gamma, \psi, \text{pk}, u, \hat{m})$   
7: **if**  $g^{f_x} h^{f_y} u^{f_z} = \gamma \cdot \text{pk}^e$  and  $u^{f_x} H_2^{f_y} H_3^{f_z} = \psi \cdot \hat{m}^e$  **then**  
8:     **return** 1  
9: **return** 0

Fig. 5:  $\Sigma$ -protocol for computing and verifying the correctness proof for partial decryptions CCA scheme.

## 6.1 Correctness and Robustness

Our IND-CCA secure threshold decryption scheme satisfies the correctness property defined in Definition 3, which follows from the completeness property of our NIZK protocol. See Section 4.2 for a more detailed discussion.

Our adaptively secure threshold Shoup-Gennaro decryption scheme satisfies the robustness property defined in Definition 7 through reasoning similar to that in Lemma 5, except that the reduction adversary additionally answers decryption oracle queries.

## 6.2 IND-TCCA Against an Adaptive Adversary

We will prove IND-TCCA of our Shoup-Gennaro based threshold decryption scheme using a sequence of attack games. Game  $\mathbf{G}_0$  is the IND-TCCA $_{\text{TD}}^A$  game in which  $\mathcal{A}$  makes queries to the decryption and encryption oracles and tries to guess the challenge bit  $\mu$ . Game  $\mathbf{G}_5$  is a game where  $\mathcal{A}$  receives a ciphertext  $\text{ctxt}$  that is independent of the challenge bit  $\mu$ . Therefore, the probability of  $\mathcal{A}$  winning game  $\mathbf{G}_5$  is exactly 1/2. For any adversary  $\mathcal{A}$  in the  $k$ -th game  $\mathbf{G}_k$ , we will use  $G_k$  to denote the event that the game outputs 1, i.e.,  $\mathcal{A}$  wins the game  $\mathbf{G}_k$ .

GAME  $\mathbf{G}_0$ : This is the security game IND-TCCA $_{\text{TD}}^A$  for our threshold decryption scheme, where the game follows the honest protocol. Here, the game provides  $\mathcal{A}$  with access to all random oracles via the standard lazy simulation technique. As in our IND-CPA analysis, we assume that  $\mathcal{A}$  always corrupts exactly  $t$  parties.

GAME  $\mathbf{G}_1$ : This game is identical to game  $\mathbf{G}_0$ , except we sample  $\alpha_h, \alpha_v, a, k \leftarrow \mathbb{Z}_p$ .  $h = g^{\alpha_h}$ ,  $v = g^{\alpha_v}$ ,  $\bar{g} = g^{ak}$ . Clearly,  $\mathbf{G}_0$  and  $\mathbf{G}_1$  are indistinguishable. Thus,  $\Pr[G_0] = \Pr[G_1]$

GAME  $\mathbf{G}_2$ : This game is identical to game  $\mathbf{G}_1$ , except that we sample  $b_j, \eta_j \leftarrow \mathbb{Z}_p$ , and program the random oracles for the  $j$ th decryption query on  $\text{ctxt}_j$  to be  $H_2(\text{ctxt}_j) := g^{b_j}$  and  $H_3(\text{ctxt}_j) := (g^{-\eta_j} (g^{b_j})^{\alpha_v})^{1/\alpha_h} \bar{u}^{1/k}$ . More precisely, whenever  $\mathcal{A}$  queries the random oracles  $H_2$  or  $H_3$  with some  $\text{ctxt}_j$  for the first time, we program  $H_2$  and  $H_3$  as described above.

Similarly, for each encryption query on message  $m_j$ , we honestly compute the encryption of  $m_j$  as per the protocol description. Then, to compute the partial decryptions of the ciphertext of  $m_j$ , we program the random oracles  $H_2$  and  $H_3$  according to the description above.

Looking ahead, this game and the following game are preparing for game  $\mathbf{G}_5$  where we “rig” the secret keys such that  $y, z$  are non-zero. We program  $H_2, H_3$  in such a way that in **Comb**,  $\hat{m} := \text{pk}^r$  and the decrypted plaintext is consistent with the honest protocol despite dishonestly chosen secret keys.

Since  $b_j, \eta_j$  are independently random,  $H_2, H_3$  are indistinguishable from uniformly random functions. Thus  $\Pr[\mathbf{G}_1] = \Pr[\mathbf{G}_2]$ .

**GAME  $\mathbf{G}_3$ :** This game is identical to game  $\mathbf{G}_2$ , except we correlate  $H_2$  and  $H_3$  by setting  $\eta_j := a \cdot b_j$ . We show the indistinguishability between game  $\mathbf{G}_2$  and game  $\mathbf{G}_3$  assuming the hardness of DDH. More formally, we prove the following:

**Lemma 11.** *Let  $\varepsilon_{\text{ddh}}$  be the advantage of breaking DDH in  $\mathbb{G}$  as defined in as defined in Assumption 1, then  $|\Pr[G_2] - \Pr[G_3]| \leq \varepsilon_{\text{ddh}} + 1/p$ .*

*Proof.* We define a distinguisher  $\mathcal{D}$  for distributions  $\mathcal{D}_0$  and  $\mathcal{D}_1$  defined in lemma 1. Let  $\{g, A, \{B_i, C_i\}_{i \in q_H}\}$  be the input to  $\mathcal{D}$ .  $\mathcal{D}$  interacts with  $\mathcal{A}$  by setting  $\bar{g} := A^k, k \leftarrow \$ \mathbb{Z}_p$ , for the  $j$ th query, either encryption or decryption query, program  $H_2(\text{ctxt}_j) := B_j, H_3(\text{ctxt}_j) := (C_j^{-1} B_j^{\alpha_v})^{1/\alpha_h} (\bar{u}_j)^{1/k}$  where  $\text{ctxt}_j$  is queried by  $\mathcal{A}$  or computed by the simulator. Lastly,  $\mathcal{D}$  outputs 1 if  $\mathcal{A}$  wins the attack game.

If  $\mathcal{D}$  receives  $\mathcal{D}_0$ ,  $\mathcal{A}$ 's view is identical to that of game  $\mathbf{G}_2$  and  $\Pr[\mathcal{D} \Rightarrow 1 | \mathcal{D}_0] = \Pr[\mathbf{G}_2]$ ; if  $\mathcal{D}$  receives  $\mathcal{D}_1$ ,  $\mathcal{A}$ 's view is identical to that of game  $\mathbf{G}_3$  and  $\Pr[\mathcal{D} \Rightarrow 1 | \mathcal{D}_1] = \Pr[\mathbf{G}_3]$ . Thus combined with Lemma 1,  $|\Pr[G_2] - \Pr[G_3]| \leq \varepsilon_{\text{ddh}} + 1/p$ .  $\square$

**GAME  $\mathbf{G}_4$ :** This game is identical to game  $\mathbf{G}_3$ , except that we replace the two real instances of NIZK proofs with simulated ones. The first is the NIZK used to prove correctness of partial decryptions described in Figure 5, and the second is the additional elements in the Shoup-Gennaro construction that can be seen as an equality of discrete logarithm NIZK proof with the modification that we also hash the ciphertext.

1. we use simulated proofs for the correctness of partial decryptions instead of actual NIZK proofs. Looking ahead, we simulate the NIZK proofs in our decryption queries so that the NIZK proofs do not leak information about the secret keys. This step is preparing for game  $\mathbf{G}_5$ .
2. we use simulated proofs for proof of discrete logarithm identities for  $\log_g u = \log_{\bar{g}} \bar{u}$ . Namely, we sample  $e, f \leftarrow \$ \mathbb{Z}_p$  then compute  $w := g^f u^{-e}, \bar{w} := \bar{g} \bar{u}^{-e}$ . Lastly, we program  $H_1(c, u, w, \bar{u}, \bar{w}) := e$ . The simulated NIZK proofs are of the correct distributions. Looking ahead, we switch to simulated proofs in this game to later argue in game  $\mathbf{G}_7$  that the proofs do not reveal any information about the discrete log of  $u$  and  $\bar{u}$ .

**Lemma 12.** *Let  $q_d$  be the upper bound on the number of combined encryption and decryption queries,  $q_{\text{FS}}$  be the number of  $H_{\text{FS}}$  queries  $\mathcal{A}$  makes, and  $q_H$  be the number of  $H_1$  queries  $\mathcal{A}$  makes, then,*

$$|\Pr[G_3] - \Pr[G_4]| \leq \varepsilon_{\text{nizk-fail}} \quad (20)$$

where  $\varepsilon_{\text{nizk-fail}} = \varepsilon_{\text{ZK}} \cdot q_d \cdot n + q_H/p^2$ .

Let  $\varepsilon_{\text{nizk-fail}}$  bound the advantage in distinguishing simulated proofs from real proofs for either the first or second NIZK instance. Following the same reasoning as Lemma 7, the probability that simulated proofs for the first NIZK instance can be distinguished from real proofs is at most  $\varepsilon_{\text{ZK}} \cdot q_d \cdot n$ . The distinguishing advantage of the second instance, following a similar zero-knowledge analysis of Lemma 4, is at most  $q_H/p^2$ . Then applying union bound, we obtain  $\varepsilon_{\text{nizk-fail}} = \varepsilon_{\text{ZK}} \cdot q_d \cdot n + q_H/p^2$ .

**GAME  $\mathbf{G}_5$ :** This game is identical to game  $\mathbf{G}_4$  except we *rig* the secret keys by sampling  $y \leftarrow \$ \mathbb{Z}_p$  and  $z := 1$ . Furthermore, we compute  $a := \alpha_h y + \alpha_v$ , where  $\bar{g} := g^{ak}$ . To help us bound  $\Pr[G_4] - \Pr[G_5]$ , we define event **Neq**: during any decryption query,  $\mathcal{A}$  queries the decryption oracle with a ciphertext,  $\text{ctxt} := (c, u, \bar{u}, e, f)$ , such that the following two conditions hold:

1. equation 18 returns 1.
2.  $u = g^r, \bar{u} = \bar{g}^{r'}, r \neq r'$ .

To show that events  $G_4$  and  $G_5$  are close, we first show that event **Neq** occurs with negligible probability, and then show that  $\Pr[G_4|\neg\text{Neq}] = \Pr[G_5|\neg\text{Neq}]$ .

We first bound  $\Pr[\text{Neq}]$  in the following lemma:

**Lemma 13.**  $\Pr[\text{Neq}] \leq q_d/p$  where  $q_d$  is the number of decryption and encryption queries.

*Proof.* First, we compute the probability that  $\mathcal{A}$  queries exactly one decryption oracle and the input ciphertext satisfies the above two conditions, i.e., check 18 returns 1 but  $r \neq r'$ . We have the following:

$$g^f = wg^{r^e}, \quad \bar{g}^f = \bar{w}\bar{g}^{r'^e} \quad (21)$$

where  $w = g^s$  and  $\bar{w} = \bar{g}^{s'}$  for some  $s, s' \in \mathbb{Z}_p$ . Equation (21) implies that  $(s - s') + e(r - r') = 0$ . Since  $r - r' \neq 0$ , there is only one  $e := -\frac{s-s'}{r-r'}$ . Since  $e$  is the output of random oracle  $H_1$ , and perfectly binds and is independent of  $r, r', s, s'$  ( $u, w, \bar{u}, \bar{w}$  are inputs to  $H_1$  therefore  $e$  is decided after  $\mathcal{A}$  chooses  $r, r', s, s'$ ), the probability of such an  $e$  occurring is  $1/p$ .

Next, we bound the probability of event **Neq**. Event **Neq** occurs if  $\mathcal{A}$  queries at least one accepted invalid ciphertext. Applying union bound, we have  $\Pr[\text{Neq}] \leq \frac{q_d}{p}$ .  $\square$

**Lemma 14.**  $|\Pr[G_4] - \Pr[G_5]| \leq q_d/p$

*Proof.* First, we show that  $\Pr[G_4|\neg\text{Neq}] = \Pr[G_5|\neg\text{Neq}]$ . To prove this, we rely on the  $H$ -coefficient technique and show that for all feasible transcripts  $\tau$ ,  $\Pr[T = \tau|\mathbf{G}_4, \neg\text{Neq}] = \Pr[T = \tau|\mathbf{G}_5, \neg\text{Neq}]$ . In other words, we show that the probability of obtaining any feasible transcript is identical in games  $\mathbf{G}_5$  and  $\mathbf{G}_6$ , conditioned on the event **Neq**.

The transcript includes all group elements, i.e.,  $g, h, v, \bar{g}$ , the public key  $\text{pk}$ , corruption set  $\mathcal{C}$ , corrupted secret keys  $\{x_i, y_i, z_i\}_{i \in [\mathcal{C}]}$ , all parties' per-party public keys  $\text{pk}_i$ , all random oracle queries,  $q_d$  ciphertexts and their corresponding partial decryption shares, and  $\mathcal{A}$ 's challenge plaintexts and their corresponding encryption.

To argue that  $\Pr[T = \tau|\mathbf{G}_4, \neg\text{Neq}] = \Pr[T = \tau|\mathbf{G}_5, \neg\text{Neq}]$ , we invoke lemma 2. We define two distributions,  $\Omega$  and  $\Psi$ . We will show that there exists a deterministic function  $f$  such that  $f(\Omega, \Psi) = T$  for both games  $\mathbf{G}_4$  and  $\mathbf{G}_5$ , provided that event **Neq** does not occur. By lemma 2, if  $\Pr[\Omega = \omega|\mathbf{G}_4] = \Pr[\Omega = \omega|\mathbf{G}_5]$ ,  $\Pr[\Psi = \psi|\mathbf{G}_4] = \Pr[\Psi = \psi|\mathbf{G}_5]$ , and  $\Omega$  is independent from  $\Psi$ , then  $\Pr[T = \tau|\mathbf{G}_4, \neg\text{Neq}] = \Pr[T = \tau|\mathbf{G}_5, \neg\text{Neq}]$ . We condition on  $\neg\text{Neq}$  because there does not exist a deterministic function  $f$  such that  $f(\Omega, \Psi) = T$  in game  $\mathbf{G}_5$ .

We define

$$\begin{aligned} \Omega &:= \{\alpha_h, \alpha_v, a, k, \text{pk}, \{x_i, y_i, z_i\}_{i \in \mathcal{C}}\} \\ \Psi &:= \{H'_1, H'_{F5}, \{b_j\}_{j \in q_H}, \{r_j, e_j, f_j\}_{j \in q_d}, r, e, f\} \end{aligned}$$

Here  $H'_1$  denotes the additional outputs of  $H_1$  not used in the computation of encryptions and  $H'_{F5}$  denotes the set of all outputs of  $H_{F5}$  queries.  $\{r_j, e_j, f_j\}_{j \in q_d}$  denote the set of randomness required for up to  $q_d$  number of encryption queries.  $r, e, f$  are the randomness required to encrypt  $\mathcal{A}$ 's challenge  $m_\mu$ . Recall that we simulate the NIZK of discrete logarithm equality in the encryption protocol by sampling  $e, f$  randomly. In both games, we have  $h := g^{\alpha_h}, v := g^{\alpha_v}, \bar{g} := g^{a^k}$ . In game  $\mathbf{G}_4$ ,  $\text{pk} := g^x$  while in game  $\mathbf{G}_5$ ,  $\text{pk} := g^{x+\alpha_h y+\alpha_v}$ . Recall in both games  $\mathbf{G}_5$  and  $\mathbf{G}_6$ ,  $\alpha_h, \alpha_v, k, H'_1, H'_{F5}, \{b_j\}_{j \in q_H}, r, s$  are sampled randomly and independently. Clearly,  $\Omega$  and  $\Psi$  are independently distributed.

Next, we show that there is a deterministic function  $f$  for both games  $\mathbf{G}_4$  and  $\mathbf{G}_5$  for any fixed  $\mathcal{A}$  queries such that  $f(\Omega, \Psi) := T$ .

- challenge response. Clearly, in both games, the challenge ciphertext is the output of a deterministic function of  $\mathcal{A}$ 's query and the values  $r, s, H_1, \text{pk}, a, k$ .
- random oracle queries. In both games, sets  $H'_1$  and  $H'_{F5}$  determines all outputs of random oracles  $H_1, H_{F5}$  respectively.  $\{b_j\}_{j \in q_H}$  contain all responses to  $H_2$  queries. Responses to  $H_3$  are computed from  $\{b_j\}_{j \in q_H}, a, \alpha_v, \alpha_h$ , and  $k$ .

- decryption query. Let  $\text{ctxt} := (c, u, \bar{u}, e, f)$  be the  $j$ th decryption query. Clearly, for both games, for  $i \in \mathcal{C}$ ,  $\hat{m}_i$  is computed deterministically from  $x_i, y_i, z_i, a, b, \alpha_v, \alpha_h$ .

Next, we show that the combined  $\hat{m}$  is identical in both games. In game  $\mathbf{G}_4$ ,  $\hat{m} = u^x = \text{pk}^r$ .

In game  $\mathbf{G}_5$ , if event **Neq** does not occur, i.e.,  $u := g^r$  and  $\bar{u} := \bar{g}^r$ , we have  $\bar{u}^{1/k} := g^{ar}$ . Then we have:

$$\begin{aligned}\hat{m} &:= u^x g^{by} (g^{-ab} g^{b\alpha_v})^{1/\alpha_h} \bar{u}^{1/k} \\ &:= u^x g^{by} g^{-b \frac{a-\alpha_v}{\alpha_h}} g^{ar} \\ &:= u^x g^{by} g^{-by} g^{ar}, \text{ by } y := \frac{a - \alpha_v}{\alpha_h} \\ &:= g^{rx + \alpha_h yr + \alpha_v} \\ &:= \text{pk}^r\end{aligned}$$

- encryption query. Let  $m_j$  be the  $j$ th encryption query. In both games, the encryption of  $m_j$  is computed according to  $r_j, e_j, f_j, \text{pk}$  where  $\text{Enc}(m_j) := (c, u, \bar{u}, e, f) := (\text{pk}^{r_j} \cdot m_j, g^{r_j}, \bar{g}^{r_j}, e_j, f_j)$ . The decryption shares are computed identically to the decryption query on  $\text{Enc}(m_j)$

Clearly  $\Pr[\Psi = \psi | \mathbf{G}_4] = \Pr[\Psi = \psi | \mathbf{G}_5]$  for all feasible  $\psi$  since  $\Psi$  is randomly sampled from the same distribution in both games.

Lastly, we show that  $\Pr[\Omega = \omega | \mathbf{G}_4] = \Pr[\Omega = \omega | \mathbf{G}_5]$  for all feasible  $\omega$ . Without loss of generality, let  $\omega := \{\alpha_h^*, \alpha_v^*, a^*, k^*, \text{pk}^*, \{x_i^*, y_i^*, z_i^*\}_{i \in \mathcal{C}}\}$ . In game  $\mathbf{G}_4$ , there is one such  $x$  such that  $\text{pk}^* = g^x$ . Since  $t$  evaluation points and the constant term uniquely defines a degree  $t$  polynomial and polynomials  $x, y, z$  are randomly and independently chosen,  $\Pr[\{x_i, y_i, z_i\}_{i \in \mathcal{C}} = \{x_i^*, y_i^*, z_i^*\}_{i \in \mathcal{C}} | x_0 = x, y_0 = 0, z_0 = 0] = 1/p^{3t}$ . Then  $\Pr[\Omega = \omega | \mathbf{G}_4] := \frac{1}{p-1} \cdot \frac{1}{p-1} \cdot \frac{1}{p} \cdot \frac{1}{p} \cdot \frac{1}{p} \cdot \frac{1}{p^{3t}}$ .

In game  $\mathbf{G}_5$ , given  $a^*, \alpha_h^*, \alpha_v^*$ , there is one such  $y$  such that  $a^* = \alpha_h^* y + \alpha_v^*$ . Then given  $\text{pk}^*, y$ , there is one such  $x$  such that  $\text{pk}^* := g^{x' + \alpha_h^* y + \alpha_v^*}$ . By the same argument as in game  $\mathbf{G}_4$ ,  $\Pr[\{x_i, y_i, z_i\}_{i \in \mathcal{C}} = \{x_i^*, y_i^*, z_i^*\}_{i \in \mathcal{C}} | x_0 = x, y_0 = y, z_0 = 1] = 1/p^{3t}$ . Then  $\Pr[\Omega = \omega | \mathbf{G}_5] := \frac{1}{p-1} \cdot \frac{1}{p-1} \cdot \frac{1}{p} \cdot \frac{1}{p} \cdot \frac{1}{p} \cdot \frac{1}{p^{3t}}$ . Therefore  $\Pr[\Omega = \omega | \mathbf{G}_4] = \Pr[\Omega = \omega | \mathbf{G}_5]$ .

Then we have  $|\Pr[G_4] - \Pr[G_5]| = |\Pr[G_4 | \text{Neq}] - \Pr[G_5 | \text{Neq}]| \cdot \Pr[\text{Neq}] \leq \Pr[\text{Neq}]$ . Then from lemma 13,  $|\Pr[G_4] - \Pr[G_5]| \leq qd/p$ .  $\square$

**GAME  $\mathbf{G}_6$ :** In this game, when  $\mathcal{A}$  challenges  $m_0, m_1$ , instead of encrypting  $\mathcal{A}$ 's challenge, we respond with an encryption that does not correlate with it. This helps bound  $|\Pr[G_0] - 1/2|$ . This game is identical to game  $\mathbf{G}_5$ , except that we sample random group elements  $\{g^\alpha, g^\beta, g^\zeta\} \leftarrow \mathbb{G}^3$ . Set  $v = g^\alpha$ . Compute  $g^a := g^{\alpha_h y} g^\alpha$ ,  $\bar{g} := (g^a)^k$ . To answer encryption query on  $m_\mu$ , compute  $u := g^\beta$ ,  $\bar{u} := (g^\beta)^{\alpha_h y k} (g^\zeta)^k$ ,  $c := (g^\beta)^x (g^\beta)^{\alpha_h y} g^\zeta \cdot m_\mu$ . For the  $j$ th decryption query, program  $\text{H}_3(\text{ctxt}) := ((g^a)^{-b} (g^\alpha)^b)^{1/\alpha_h} \bar{u}^{1/k}$ .

**Lemma 15.** Let  $\varepsilon_{\text{ddh}}$  be the advantage of breaking DDH in  $\mathbb{G}$  as defined in as defined in Assumption 1, then  $|\Pr[G_5] - \Pr[G_6]| \leq \varepsilon_{\text{ddh}} + qd/p$ .

*Proof.*  $\mathcal{A}$  distinguishes game  $\mathbf{G}_5$  from game  $\mathbf{G}_6$  either by making a decryption query with a ciphertext that satisfies the two conditions laid out in game  $\mathbf{G}_5$ , i.e., inducing event **Neq** or, by breaking the DDH assumption in  $\mathbb{G}$ , since  $\mathcal{A}$  cannot query the decryption query on  $\text{ctxt}_\mu$ . We have

$$\begin{aligned}|\Pr[G_5] - \Pr[G_6]| &= |\Pr[G_5 | \neg \text{Neq}] - \Pr[G_6 | \neg \text{Neq}]| \cdot \Pr[\neg \text{Neq}] \\ &\quad + |\Pr[G_5 | \text{Neq}] - \Pr[G_6 | \text{Neq}]| \cdot \Pr[\text{Neq}] \\ &\leq |\Pr[G_5 | \neg \text{Neq}] - \Pr[G_6 | \neg \text{Neq}]| + \Pr[\text{Neq}]\end{aligned}$$

Next, we show that  $|\Pr[G_5 | \neg \text{Neq}] - \Pr[G_6 | \neg \text{Neq}]| \leq \varepsilon_{\text{ddh}}$ . We will prove this by building an adversary against the DDH assumption,  $\mathcal{A}_{\text{ddh}}$ , from  $\mathcal{A}$ .  $\mathcal{A}_{\text{ddh}}$  receives a tuple  $(g^\alpha, g^\beta, g^\zeta)$  and interacts with  $\mathcal{A}$  as described. If  $\mathcal{A}_{\text{ddh}}$ 's input is a DDH tuple, then  $\mathcal{A}$  plays against game  $\mathbf{G}_5$ . If  $\mathcal{A}_{\text{ddh}}$ 's input is a random tuple, then  $\mathcal{A}$  plays against game  $\mathbf{G}_6$ . We have the following:

$$\Pr[\mathcal{A}_{\text{ddh}}(g^\alpha, g^\beta, g^\zeta) = 1 | \zeta = \alpha\beta] = \Pr[G_5] \quad (22)$$

$$\Pr[\mathcal{A}_{\text{ddh}}(g^\alpha, g^\beta, g^\zeta) = 1 | \zeta \leftarrow \mathbb{Z}_p] = \Pr[G_6] \quad (23)$$

Hence, combined with Lemma 13,  $\mathcal{A}_{\text{ddh}}$ 's advantage is  $|\Pr[G_5] - \Pr[G_6]| \leq \varepsilon_{\text{ddh}} + q_d/p$ .  $\square$

In the last part of our analysis, we will prove that  $\mathcal{A}$  wins in game  $\mathbf{G}_6$  with probability  $1/2$ .

**Lemma 16.**  $\Pr[G_6] = 1/2$ .

*Proof.* The only part of  $\mathcal{A}$ 's view that depends on the hidden bit  $\mu$  is  $c$ . In game  $\mathbf{G}_6$ ,  $\mathcal{A}$  receives  $c = (g^\beta)^x (g^\beta)^{\alpha_h y} g^\zeta \cdot m_\mu$  from the game. Since  $\beta, \zeta \leftarrow \mathbb{Z}_p$ ,  $c$  is uniformly randomly distributed in  $\mathbb{G}$  for both  $m_0$  and  $m_1$ , and is independent of  $\mu$ . Thus:

$$\Pr[\text{ctxt}|\mu = 0] = \Pr[\text{ctxt}|\mu = 1] = \frac{1}{p^2}$$

Thus, we have

$$\Pr[\mathcal{A} = 1|\mu = 0] = \Pr[\mathcal{A} = 1|\mu = 1] = \Pr[G_6] = 1/2. \quad \square$$

We now prove the following main theorem.

**Theorem 4 (Adaptively IND-TCCA Secure Threshold Decryption).** *Let  $\lambda$  be the security parameter, and let  $(\mathbb{G}, p, g) \leftarrow \text{GGen}(1^\lambda)$  be a group of prime order  $p$ . For any  $n \in \mathbb{N}$  and  $t < n$ , assuming the hardness of DDH in  $\mathbb{G}$  and the random oracle model, any PPT adversary making at most  $q_d$  decryption queries,  $q_H$  random oracle queries to  $H$ , and  $q_{FS}$  random oracle queries to  $H_{FS}$  has guessing advantage in the  $\text{IND-TCCA}_{\text{TD}}^A$  game (Figure 1) for our scheme in Figure 2 of at most  $\varepsilon_{\text{IND-TCCA}}$ , where*

$$\varepsilon_{\text{IND-TCCA}} \leq 2\varepsilon_{\text{ddh}} + \frac{q_{FS} \cdot q_d \cdot n}{p^2} + \frac{q_H}{p^2} + \frac{1}{p} + 2\frac{q_d}{p} \quad (24)$$

where  $\varepsilon_{\text{ddh}}$  is the advantage an adversary in breaking DDH in  $\mathbb{G}$ .

*Proof.* From the above series of games and lemma 16, we get

$$|\Pr[G_0] - \frac{1}{2}| \leq 2\varepsilon_{\text{ddh}} + \frac{q_{FS} \cdot q_d \cdot n}{p^2} + \frac{q_H}{p^2} + \frac{1}{p} + 2\frac{q_d}{p} \leq \text{negl}(\lambda) \quad (25)$$

which is  $\mathcal{A}$ 's guessing advantage against our threshold encryption scheme.  $\square$

## 7 Conclusion

In this paper, we presented two new adaptively secure non-interactive threshold decryption schemes. The first one is based on the standard ElGamal decryption scheme and achieves CPA security. The second one is based on the Shoup-Genarro threshold decryption scheme and achieves CCA security. Our scheme makes minimal changes to their static counterparts, maintains the scheme's simplicity, and is compatible with its encryption. Both schemes are proven adaptively secure assuming the hardness of DDH and the random oracle model.

**Acknowledgments.** This work is funded in part by the National Science Foundation award #2240976.

## References

- BBH06. Dan Boneh, Xavier Boyen, and Shai Halevi. Chosen ciphertext secure public key threshold encryption without random oracles. In *Cryptographers' Track at the RSA Conference*, pages 226–243. Springer, 2006.
- BS23. Dan Boneh and Victor Shoup. A graduate course in applied cryptography. *Draft 0.6*, 2023.
- CG99. Ran Canetti and Shafi Goldwasser. An efficient threshold public key cryptosystem secure against adaptive chosen ciphertext attack. In *International Conference on the Theory and Applications of Cryptographic Techniques*, pages 90–106. Springer, 1999.

- CGJ<sup>+</sup>99. Ran Canetti, Rosario Gennaro, Stanisław Jarecki, Hugo Krawczyk, and Tal Rabin. Adaptive security for threshold cryptosystems. In *Advances in Cryptology—CRYPTO’99: 19th Annual International Cryptology Conference Santa Barbara, California, USA, August 15–19, 1999 Proceedings 19*, pages 98–116. Springer, 1999.
- CGS97. Ronald Cramer, Rosario Gennaro, and Berry Schoenmakers. A secure and optimally efficient multi-authority election scheme. In Walter Fumy, editor, *Advances in Cryptology — EUROCRYPT ’97*, pages 103–118, Berlin, Heidelberg, 1997. Springer Berlin Heidelberg.
- CS14. Shan Chen and John Steinberger. Tight security bounds for key-alternating ciphers. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 327–350. Springer, 2014.
- Des88. Yvo Desmedt. Society and group oriented cryptography: A new concept. In *Advances in Cryptology—CRYPTO’87: Proceedings 7*, pages 120–127. Springer, 1988.
- Des93. Yvo Desmedt. Threshold cryptosystems. In *Advances in Cryptology—AUSCRYPT’92: Workshop on the Theory and Application of Cryptographic Techniques Gold Coast, Queensland, Australia, December 13–16, 1992 Proceedings 3*, pages 1–14. Springer, 1993.
- DK05. Yevgeniy Dodis and Jonathan Katz. Chosen-ciphertext security of multiple encryption. In *Theory of Cryptography Conference*, pages 188–209. Springer, 2005.
- DLN<sup>+</sup>21. Julien Devevey, Benoît Libert, Khoa Nguyen, Thomas Peters, and Moti Yung. Non-interactive cca2-secure threshold cryptosystems: Achieving adaptive security in the standard model without pairings. In Juan A. Garay, editor, *Public-Key Cryptography – PKC 2021*, pages 659–690, Cham, 2021. Springer International Publishing.
- DR24. Sourav Das and Ling Ren. Adaptively secure bls threshold signatures from ddh and co-cdh. In *Annual International Cryptology Conference*, pages 251–284. Springer, 2024.
- ElG85. Taher ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE transactions on information theory*, 31(4):469–472, 1985.
- FMY99a. Yair Frankel, Philip MacKenzie, and Moti Yung. Adaptively-secure distributed public-key systems. In *European Symposium on Algorithms*, pages 4–27. Springer, 1999.
- FMY99b. Yair Frankel, Philip MacKenzie, and Moti Yung. Adaptively-secure optimal-resilience proactive rsa. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 180–194. Springer, 1999.
- FS86. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*, pages 186–194. Springer, 1986.
- JL00. Stanisław Jarecki and Anna Lysyanskaya. Adaptively secure threshold cryptography: Introducing concurrency, removing erasures. In *Advances in Cryptology—EUROCRYPT 2000: International Conference on the Theory and Application of Cryptographic Techniques Bruges, Belgium, May 14–18, 2000 Proceedings 19*, pages 221–242. Springer, 2000.
- KL07. Jonathan Katz and Yehuda Lindell. *Introduction to modern cryptography: principles and protocols*. Chapman and hall/CRC, 2007.
- Lin01. Yehuda Lindell. Parallel coin-tossing and constant-round secure two-party computation. Cryptology ePrint Archive, Paper 2001/107, 2001.
- LY12. Benoît Libert and Moti Yung. Non-interactive cca-secure threshold cryptosystems with adaptive security: New framework and constructions. In Ronald Cramer, editor, *Theory of Cryptography*, pages 75–93, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- LY13. Benoît Libert and Moti Yung. Adaptively secure non-interactive threshold cryptosystems. *Theoretical Computer Science*, 478:76–100, 2013.
- MK02. Philip Mac Kenzie. An efficient two-party public key cryptosystem secure against adaptive chosen ciphertext attack. In Yvo G. Desmedt, editor, *Public Key Cryptography — PKC 2003*, pages 47–61, Berlin, Heidelberg, 2002. Springer Berlin Heidelberg.
- NR04. Moni Naor and Omer Reingold. Number-theoretic constructions of efficient pseudo-random functions. *Journal of the ACM (JACM)*, 51(2):231–262, 2004.
- Pat08. Jacques Patarin. The “coefficients h” technique. In *International Workshop on Selected Areas in Cryptography*, pages 328–345. Springer, 2008.
- Ped91. Torben Prys Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Annual international cryptology conference*, pages 129–140. Springer, 1991.
- SG98. Victor Shoup and Rosario Gennaro. Securing threshold cryptosystems against chosen ciphertext attack. In *International Conference on the Theory and Applications of Cryptographic Techniques*, pages 1–16. Springer, 1998.

- SG02. Victor Shoup and Rosario Gennaro. Securing threshold cryptosystems against chosen ciphertext attack. *J. Cryptol.*, 15(2):75–96, January 2002.
- Sha79. Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
- Wee11. Hoeteck Wee. Threshold and revocation cryptosystems via extractable hash proofs. In Kenneth G. Paterson, editor, *Advances in Cryptology – EUROCRYPT 2011*, pages 589–609, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.